

VOLE-in-the-Head Signatures from Subfield Bilinear Collisions

Janik Huth^{1,2} and Antoine Joux¹

¹ CISP - Helmholtz Center for Information Security, Saarbrücken, Germany
janik.huth@cispa.de, joux@cispa.de

² Saarland University, Saarbrücken, Germany

Abstract. In this paper, we introduce a new signature scheme based on the subfield bilinear collision problem published at Crypto 2024. We use techniques based on vector oblivious linear evaluation (VOLE) to significantly improve both the running time and the signature size of the scheme compared to the MPC-in-the-Head version. Furthermore, we introduce the correlated GGM forest construction, which is a generic method to correlate several GGM trees across multiple rounds of the signature scheme. This construction combines the correlated tree derivation with the hypercube folding in a layered construction.

Keywords: Post-quantum signatures, ZK protocols, VOLE-in-the-Head

1 Introduction

The *Multi-Party Computation in the Head* (MPCitH) paradigm was first introduced in [22] as a new approach to design *Zero-Knowledge* (ZK) protocols. This approach leverages a multiparty protocol checking an NP-relation $\mathcal{R}(x, w)$ to construct a ZK protocol where a prover $\mathcal{P}$ convinces a verifier $\mathcal{V}$ that she knows a witness w for a given (public) value of x , without disclosing any information about w . In the MPCitH protocol, the prover emulates the execution of the underlying MPC protocol checking the relation being proven. The verifier $\mathcal{V}$ then checks parts of this execution in a way that allows him to verify the correctness of the relation while preserving the secrecy of the witness w . As with other identification protocols, those based on MPCitH can be turned into signature schemes by using the Fiat-Shamir transform [18]. In recent years, a large number of MPCitH-based signature schemes have been proposed [16,17,9,3,15]. These schemes are quite efficient in terms of speed, but the corresponding signatures remain relatively large compared to classical signature schemes such as [27,29,2]. In order to limit this problem and reduce the size of signatures, Baum et al. introduced an additional twist on MPCitH, called *VOLE-in-the-Head* [5], which relies on *vector oblivious linear evaluation* (VOLE) and uses a

non-interactive version of the SoftspokenOT technique [28] within the QuickSilver proof system [30]. This technique significantly reduces the signature size and, as a side bonus, also improves the running times of signature schemes, for example, see [12,14,10].

1.1 Our Contribution

In this paper, we consider the *subfield bilinear collision* (SBC) problem proposed at Crypto 2024 [21] and develop a *VOLE-in-the-Head* signature scheme based on this problem. In [21], it already appeared that the structure of the problem was highly suited to the MPCitH paradigm, leading to a simple and quite compact signature scheme. In the present paper, we show that the structure of the SBC problem also works very well in the context of *VOLE-in-the-Head*. In fact, because of this, we do not need to use the full extent of the functionalities presented in [5]. Instead, we reconstruct a specific version of VOLEitH from scratch dedicated to the SBC problem in order to obtain a simplified scheme.

In MPCitH based schemes, the use of puncturable PRFs (PPRFs) based on GGM trees [19] is common to reduce the communication cost of the scheme. We introduce a new method to correlate multiple such GGM trees, which we call a *correlated GGM forest*, by using a layered construction of two state-of-the-art techniques from MPCitH protocols: the correlated GGM tree derivation [20] and the hypercube folding technique [1]. This method allows us to correlate the values obtained by the leaves of GGM trees between multiple rounds of the identification protocol inside the signature scheme, while having the same PPRF key size as state-of-the-art protocols. This is a general construction and is not tied to the use of the SBC problem, and can therefore be used for any multi-round MPCitH signature scheme.

The correlated forest can also be used in VOLEitH protocols. However, if we want to correlate values with a size larger than the internal nodes used in the GGM trees, we can derive them from the leaves of the tree. In that case, we need to include a consistency check, which convinces the verifier that the values obtained between multiple rounds of the protocol coincide. A common method to perform this (probabilistic) consistency check is known as the SoftSpokenOT technique [28] and is used in many VOLEitH schemes based on the transformation by Baum et al. [5]. In the context of the SBC problem, we simplify the consistency check in Section 3.4 and include it in the signature scheme.

Using the correlated GGM forest and this simplified consistency check, we introduce the identification scheme for the SBC problem based on VOLEitH and use the Fiat-Shamir transformation to obtain a signature scheme. To further reduce the signature size, we introduce an alternative commitment to the cGGM forest called *MinMax commitment* in Section 6.1.

Compared to the SBC signature scheme based on MPCitH from [21], our approach leads to improvements in both signature size and running time. For a security level of $\lambda = 128$ bits, our smallest signature has a size of only 2 658 B, compared to 3 766 B for the smallest signature from [21].

2 Preliminaries

2.1 Notations

We denote the target security level of our scheme by λ . For any positive integer $m \in \mathbb{Z}^+$, we denote the set $\{0, \dots, m-1\}$ by $[m]$. For $m > 1$, we denote the set $\{1, \dots, m-1\}$ by $[m]^*$. Let S be a finite set. The notation $s \xleftarrow{\$} S$ indicates that s is sampled uniformly at random from S .

Let $\mathbb{F}$ be a finite field. We denote the polynomial ring over $\mathbb{F}$ with indeterminate X by $\mathbb{F}[X]$. We indicate vectors using arrow notation, e.g., $\vec{x}$. Coordinates of $\vec{x}$ are denoted x_i , with indices starting at 0.

We denote the dot product of two vectors $\vec{u} \in \mathbb{F}^n$ and $\vec{x} \in \mathbb{F}^n$ by $\vec{u} \cdot \vec{x} = \sum_{i=0}^{n-1} u_i x_i$. Let $\vec{a} \in \mathbb{F}^{n_1}$ and $\vec{b} \in \mathbb{F}^{n_2}$ be two vectors of dimensions n_1 and n_2 , respectively. We denote the vector of dimension $n_1 + n_2$ resulting from the concatenation of the coordinates of $\vec{a}$ and $\vec{b}$ by $(\vec{a}, \vec{b})$.

Additionally, we place ourselves in the random oracle model and consider every hash function used in this paper as a random oracle. In particular, the notation $\mathcal{H}$ always refers to a random oracle.

A function $\mu: \mathbb{N} \rightarrow \mathbb{R}$ is called *negligible* if for every positive polynomial p , there exists an integer $N = N_p > 0$ such that for all $x > N_p$, we have $|\mu(x)| < 1/p(x)$. We denote an (arbitrary) negligible function in λ by $\text{negl}(\lambda)$ and a polynomial function in λ by $\text{poly}(\lambda)$.

Let $t, \varepsilon: \mathbb{N} \rightarrow \mathbb{R}$. We say that two distributions $\{D_\lambda\}_\lambda$ and $\{E_\lambda\}_\lambda$ indexed by λ are (t, ε) indistinguishable, if for any algorithm $\mathcal{A}$ with running time at most $t(\lambda)$ we have that

$$|\Pr[\mathcal{A}^{D_\lambda}(\cdot) = 1] - \Pr[\mathcal{A}^{E_\lambda}(\cdot) = 1]| \leq \varepsilon(\lambda).$$

The two distributions are *computationally indistinguishable* if $\varepsilon \in \text{negl}(\lambda)$ and $t \in \text{poly}(\lambda)$.

Definition 1 (PRG). A deterministic function $G: \{0, 1\}^\lambda \rightarrow \{0, 1\}^m$ is a (t, ε) -secure pseudorandom generator (PRG) if

1. $\lambda < m$ and
2. the distributions

$$\{G(x) | x \leftarrow \{0, 1\}^\lambda\} \text{ and } \{r | r \leftarrow \{0, 1\}^m\}$$

are (t, ε) indistinguishable.

2.2 The SBC Problem

The *subfield bilinear collision* (SBC) problem was introduced in [21]. We recall it using the same notation as in [21]:

Definition 2. Let q be a prime power and let k, n be positive integers.

- *Problem instance:* Two vectors $\vec{u}, \vec{v} \in (\mathbb{F}_{q^k})^n$, which are linearly independent over $\mathbb{F}_{q^k}$.
- *Solution:* Two vectors $\vec{x}, \vec{y} \in (\mathbb{F}_q)^n$, linearly independent over $\mathbb{F}_q$, that satisfy the relation:

$$(\vec{u} \cdot \vec{x}) (\vec{v} \cdot \vec{y}) = (\vec{u} \cdot \vec{y}) (\vec{v} \cdot \vec{x}). \quad (1)$$

We denote such an instance of the SBC problem by $\text{SBC}[\vec{u}, \vec{v}]$ and write $(\vec{x}, \vec{y}) \in \text{SBC}[\vec{u}, \vec{v}]$ when the vectors $\vec{x}, \vec{y} \in (\mathbb{F}_q)^n$ are a solution of the instance specified by $\vec{u}$ and $\vec{v}$.

Remark 1. In Equation (1), we use the canonical embedding from $\mathbb{F}_q$ to $\mathbb{F}_{q^k}$. Therefore, all operations are well-defined. As mentioned in [21], it is simpler and preferable to use normalized solutions for the SBC problem of the form $\vec{x}' = (\vec{x}, 1, 0)$ and $\vec{y}' = (\vec{y}', 0, 1)$ for $\vec{x}, \vec{y} \in (\mathbb{F}_q)^{n-2}$. Furthermore, [21] shows how to transform a regular solution into a normalized one using simple linear algebra. In the normalized case, we use the notation $(\vec{x}', \vec{y}') \in \text{NSBC}[\vec{u}, \vec{v}]$ to denote a solution.

Generating instances with a planted solution. In order to design identification and signature schemes from the SBC problem, we need the ability to efficiently create instances together with a solution. The following method from [21] solves this issue.

Given q, k and n , we can generate an instance of the SBC problem together with a normalized solution in the following way: Choose uniformly at random two vectors $\vec{x}, \vec{y} \in (\mathbb{F}_q)^{n-2}$, and let $\vec{x}' := (\vec{x}, 1, 0)$ and $\vec{y}' := (\vec{y}, 0, 1)$. Then randomly select the vector coordinates $\vec{u} \in (\mathbb{F}_{q^k})^n$ and the first $n-1$ coordinates $v_0, \dots, v_{n-2} \in \mathbb{F}_{q^k}$ of $\vec{v}$. Finally, compute the missing coordinate v_{n-1} as:

$$v_{n-1} = \frac{(\vec{u} \cdot \vec{y}') \left(\sum_{i=0}^{n-2} v_i x'_i \right) - (\vec{u} \cdot \vec{x}') \left(\sum_{i=0}^{n-2} v_i y'_i \right)}{y'_{n-1} (\vec{u} \cdot \vec{x}')}.$$

If the denominator $y'_{n-1} (\vec{u} \cdot \vec{x}')$ happens to be zero, the computation of v_{n-1} fails. This occurs with negligible probability. In this case, we can restart the selection of $\vec{u}$ and $\vec{v}$. Similarly, in the very unlikely event that $\vec{u}, \vec{v}$ are linearly dependent, we select new vectors instead. The construction guarantees that $(\vec{x}', \vec{y}') \in \text{NSBC}[\vec{u}, \vec{v}]$.

Hardness of SBC. Following the heuristic analysis from [21], the NSBC problem is hard for $n \approx \frac{k}{2}$. Note that, as mentioned in [21], any advance concerning the SBC problem could be leveraged into a faster bilinear descent step for the computation of discrete logarithms in small characteristic finite fields [23].

To design our scheme and to ease comparison, we follow the parameters suggested in [21] for a security level of $\lambda = 128$ bits, namely we use $q = 2, n = 130$ and $k = 257$.

2.3 The MPC-in-the-Head Paradigm

The construction of a ZK proof in [21] relies on the MPCitH paradigm, which was introduced in [22]. We first define notations for basic secure multiparty computations, which we use throughout this paper. We use additive sharings of finite field elements and finite field vectors. Let N be the number of parties. An N -sharing of a finite field element $x \in \mathbb{F}$ is an N -tuple

$$[[x]] = (x^{[0]}, \dots, x^{[N-1]})$$

such that

$$x = \sum_{i=0}^{N-1} x^{[i]} \quad (\text{in } \mathbb{F}).$$

We refer to $x^{[i]}$ as the i -th *share* of x . Similarly, we use the notation

$$[[\vec{x}]] = (\vec{x}^{[0]}, \dots, \vec{x}^{[N-1]})$$

for a sharing of a finite field vector $\vec{x} \in \mathbb{F}^n$. In our setting, each (virtual) party receives one of the N shares.

A sharing of a given value x is usually obtained by computing $N - 1$ random values $x^{[0]}, \dots, x^{[N-2]}$, and then setting $x^{[N-1]} = x - \sum_{i=0}^{N-2} x^{[i]} \mod |\mathbb{F}|$. In this paper, we often use a random oracle to derive the shares appearing in the protocol. In this situation, each share can be viewed as a random value, and summed together, they would share a random value. Thus, to obtain a sharing of the desired value, we need an *offset* value δ_x , computed as

$$\delta_x = x - \sum_{i=0}^{N-1} x^{[i]}.$$

We can then reconstruct x using the formula

$$x = \delta_x + \sum_{i=0}^{N-1} x^{[i]}.$$

This can also be viewed as an (exact) sharing of x between $N + 1$ parties. By introducing these offset values, we can use a random oracle to derive sharings if needed and simplify the notation used in our protocol description.

Hypercube technique. A prevalent tool to improve the efficiency of MPCitH-based schemes is the hypercube technique introduced in [1]. Let $N = 2^D$ be the number of parties. This approach transforms one instance of the protocol with $N = 2^D$ parties into D instances of the protocol, each involving only 2 parties, while maintaining the same total soundness error as the original protocol. Let the j -th bit of the binary decomposition of an integer i be $B_j(i)$. The process of

converting one protocol instance with $N = 2^D$ parties into D instances with 2 parties is referred to as *folding* of the shares. Consider the sharing of an element x between N parties of the form

$$x = \sum_{i=0}^{N-1} x^{\llbracket i \rrbracket}.$$

For any fixed value $j \in [D]$, we see that

$$\begin{aligned} x &= \sum_{B_j(i)=0} x^{\llbracket i \rrbracket} + \sum_{B_j(i)=1} x^{\llbracket i \rrbracket} \\ &= x_j^{\llbracket 0 \rrbracket} + x_j^{\llbracket 1 \rrbracket} \end{aligned}$$

is a sharing of x between two parties. In the hypercube technique, D such protocols between two parties are run in parallel, each with a soundness error of $\frac{1}{2}$. We use the notation $(x_j^{\llbracket 0 \rrbracket}, x_j^{\llbracket 1 \rrbracket})_{j \in [D]} \leftarrow \mathbf{Folding}(x^{\llbracket 0 \rrbracket}, \dots, x^{\llbracket N-1 \rrbracket})$ to denote this construction. For efficiency, we use the fast folding algorithm with a running time of $\mathcal{O}(N)$ described in [21, Section 6] in our implementation.

The key point of the hypercube technique is that the choice of one missing share among N is equivalent to the (independent) choice of one missing share in each derived binary sharing. Note that the correspondence between the missing positions simply comes from the binary decomposition of the missing index among N . Furthermore, the derived binary sharings are mutually independent.

2.4 Puncturable PRFs

To reduce communication costs, the N -sharings appearing in the construction are based on the use of *puncturable pseudorandom functions* (PPRFs). More specifically, we use the classical PPRF based on GGM trees [19], which is mentioned for example in [11,25]. Starting from a root seed, the idea is to build a binary tree with $N = 2^D$ leaves by recursively applying a length-doubling *pseudorandom generator* (PRG) to obtain the left and right children of each node. To reveal all N leaves of such a binary tree except one leaf $i^* \in [N]$, the idea is to reveal all the nodes of the siblings of the path from the root seed to the leaf i^* . Using this method, all leaves except i^* can be reconstructed by communicating $\log_2(N) = D$ nodes instead of $N - 1$ leaves.

In our scheme, we use *correlated* GGM (or cGGM) trees, as introduced in [20]. We extend the cGGM trees to construct the correlated forest in Section 3.1. The idea of regular cGGM trees is to build a binary tree where the sum of all nodes on each level is preserved as the tree is being constructed. In order to do that, it suffices to modify the length-doubling PRG so that the sum of the labels of the left and right descendants of any node equals the label of the node itself. Given the (salted) hash function $\mathcal{H}$ and any node T , this is easily achieved by setting the left child of T to $\mathcal{H}(T)$ and the right child of T to $T - \mathcal{H}(T)$. In the binary case $q = 2$, the right child is rewritten as $T \oplus \mathcal{H}(T)$, and we speak of a

XOR-preserving GGM tree. Note that correlated GGM trees have a limitation. Namely, the size of the correlated value is limited by the size of the inner nodes of the tree, usually equal to λ bits. As a consequence, if the tree is used to share more than λ bits between the parties, it is necessary to compute and transmit offsets for the values beyond the first λ bits. Of course, this constraint of having offsets does not apply to purely random values shared between the parties.

In order to improve the running time of the scheme for $\lambda = 128$, we use a cGGM tree construction based on the AES block cipher instead of a salted hash function. This technique was introduced in [13] and takes full advantage of the AES instruction set available on many recent CPU architectures. In the standard GGM, we need two keys K_0 and K_1 , the left child is set to $T \oplus \text{AES}_{K_0}(T)$ and the right child is set to $T \oplus \text{AES}_{K_1}(T)$. It is important to note that a direct application of AES would not lead to a secure PPRF construction because decryption would allow one to climb back to the root node. In order to achieve a XOR-preserving tree, we still use two keys and define the left child to be $\text{AES}_{K_0}(T) \oplus \text{AES}_{K_1}(T)$, which leads to a right child of $T \oplus \text{AES}_{K_0}(T) \oplus \text{AES}_{K_1}(T)$.

3 Identification Protocol Based on SBC

Consider an NSBC instance $\text{NSBC}[\vec{u}, \vec{v}]$ for $\vec{u}, \vec{v} \in (\mathbb{F}_{q^k})^n$. Based on the cGGM tree construction described in Section 2.4, we build an interactive Zero-Knowledge proof of knowledge of a solution $(\vec{x}', \vec{y}') \in \text{NSBC}[\vec{u}, \vec{v}]$ between a prover $\mathcal{P}$ and a verifier $\mathcal{V}$. As previously explained, the normalized vectors $\vec{x}'$ and $\vec{y}'$ are written in the form $\vec{x}' = (\vec{x}, 1, 0)$ and $\vec{y}' = (\vec{y}, 0, 1)$ for $\vec{x}, \vec{y} \in (\mathbb{F}_q)^{n-2}$. For simplicity, we only consider the case $q = 2$ for this construction. In other words, the secret key of our scheme consists of the $2n - 4$ bits forming $\vec{x}$ and $\vec{y}$.

3.1 Correlating Multiple cGGM Trees

To design our signature scheme, we need multiple (different) sharings of the vectors $\vec{x}$ and $\vec{y}$ between 2 parties in the hypercube setting. More precisely, the number of binary sharings can never be lower than the desired security level λ . Furthermore, depending on the details of the scheme and of its security proof, a few extra sharings might be needed.

In particular, we need to be able to construct a large number of (independent) sharings of $\vec{x}$ between 2 parties. When doing that, it is essential to ensure that the same value of $\vec{x}$ is used everywhere. With a standard cGGM tree with $N = 2^D$ parties, we obtain $\log_2(N) = D$ such sharings from the hypercube folding. And, by construction, both the prover and verifier have the guarantee that all sharings are for the same vector $\vec{x}$. Unfortunately, N cannot be arbitrarily large, since it would make the computation costs too high. As a consequence, repetition is needed to obtain a large enough number of sharings. Using τ repetitions, we obtain a total of $\tau \cdot D$ binary sharings, and to ensure the desired security level, we require $\tau \cdot D \geq \lambda$.

In this section, we introduce a technique that allows us to guarantee the equality of the vector $\vec{x}$ across several distinct cGGM trees. This is achieved by a layered construction: first, an initial cGGM tree is built, followed by the hypercube folding, which is used to initialize a second level consisting of τ distinct cGGM trees.

First, we construct the initial cGGM tree, which we also call the pre-tree, that contains 2^τ leaves, where every level sums to $\vec{x}$, using a bitwise XOR on λ bits. To initialize this tree, we generate a random initial value $R_{\vec{x}}$ and set the two sibling elements on the second level of the tree as $R_{\vec{x}}$ and $R_{\vec{x}} \oplus \vec{x}$. The root node is unused and can remain unassigned. The XOR of these two nodes is $\vec{x}$ by design. From them, we compute the next levels of a XOR-preserving cGGM tree $T_{\vec{x}_{\text{pre}}}$ with 2^τ leaves. We call this functionality the generation of the pre-tree and denote it by $T_{\vec{x}_{\text{pre}}} \leftarrow \text{cGGM}(R_{\vec{x}}, R_{\vec{x}} \oplus \vec{x}, 2^\tau)$. If we use a salted hash function denoted by $\mathcal{H}^{\text{salt}}(\text{msg}) := \mathcal{H}(\text{salt} \parallel \text{msg})$ for a random value $\text{salt} \leftarrow \{0, 1\}^{2^\lambda}$ to derive the internal nodes of the tree, we use the subscript $\text{cGGM}_{\text{salt}}$ instead. Let $(\vec{x}^{\llbracket 0 \rrbracket}, \dots, \vec{x}^{\llbracket 2^\tau - 1 \rrbracket})$ be the leaves of $T_{\vec{x}_{\text{pre}}}$.

Using the hypercube folding for these leaves, we obtain τ (pre) sharings of $\vec{x}$ between two parties of the form $\llbracket \vec{x} \rrbracket_j = (\vec{x}^{\llbracket 0 \rrbracket_j}, \vec{x}^{\llbracket 1 \rrbracket_j})$ for $j \in [\tau]$ by using the function $\text{Folding}(\vec{x}^{\llbracket 0 \rrbracket}, \dots, \vec{x}^{\llbracket 2^\tau - 1 \rrbracket})$. By construction, we have that $\vec{x}^{\llbracket 0 \rrbracket_j} \oplus \vec{x}^{\llbracket 1 \rrbracket_j} = \vec{x}$ for $j \in [\tau]$.

We can use each of these sharings to initialize a second level containing τ new correlated trees $T_{\vec{x}_j}$ for $j \in [\tau]$. Each of these trees can then be fully computed with the cGGM technique. We thus obtain τ trees $T_{\vec{x}_0}, \dots, T_{\vec{x}_{\tau-1}}$ with 2^D leaves each using the function $T_{\vec{x}_j} \leftarrow \text{cGGM}(\vec{x}^{\llbracket 0 \rrbracket_j}, \vec{x}^{\llbracket 1 \rrbracket_j}, 2^D)$ for each $j \in [\tau]$. The leaves of tree $T_{\vec{x}_j}$ are denoted by $(\vec{x}_j^{\llbracket 0 \rrbracket}, \dots, \vec{x}_j^{\llbracket 2^D - 1 \rrbracket})$. By using the hypercube folding on the leaves of each of these trees again, we obtain D sharings of $\vec{x}$ between 2 parties for each of the τ trees. We denote this by $(\vec{x}_j^{\llbracket 0 \rrbracket_i}, \vec{x}_j^{\llbracket 1 \rrbracket_i})_{i \in [D]} \leftarrow \text{Folding}(\vec{x}_j^{\llbracket 0 \rrbracket}, \dots, \vec{x}_j^{\llbracket 2^D - 1 \rrbracket})$ for $j \in [\tau]$. We call the family of cGGM trees containing $T_{\vec{x}_{\text{pre}}}$ and $T_{\vec{x}_0}, \dots, T_{\vec{x}_{\tau-1}}$ a *correlated GGM forest*.

Since we used XOR preserving cGGM trees on both layers of our construction, this is guaranteed to yield a total of $\tau \cdot D$ hypercube sharings of the same value $\vec{x}$.

To simplify the notation, we renumber the $\tau \cdot D$ sharings obtained in this way and simply write them as:

$$\vec{x}_m^{\llbracket 0 \rrbracket} \oplus \vec{x}_m^{\llbracket 1 \rrbracket} = \vec{x} \quad \text{for each } m \in [\tau \cdot D].$$

In addition, by using a pseudorandom generator to expand the leaves of each tree $T_{\vec{x}_j}$ into more than λ bits, it is easy to produce shares for additional values in each tree. Note, however, that these extra values are not correlated. As a consequence, they might require offsets and extra precautions to prove equality across trees.

To illustrate the idea of the construction for the correlated GGM forest, we include a diagram of the resulting structure for a small depth tree example in Figure 1.

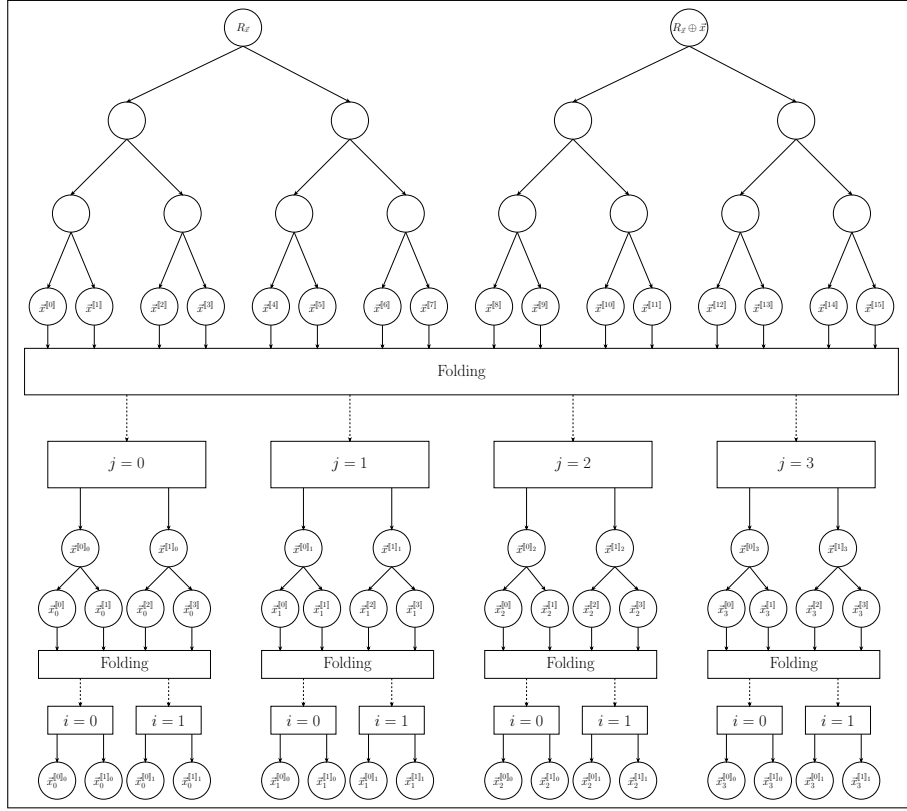


Fig. 1. Illustration of the correlated GGM forest for $\tau = 4$ and $D = 2$ resulting in 8 hypercube sharings of $\vec{x}$.

Punctured keys for the correlated GGM forest. When giving out the punctured key for a correlated GGM forest, we need to do it in a way that guarantees to the verifier that the construction was properly used. Thus, it is not sufficient to independently give a punctured key for each tree from the second level. Indeed, such an opening would be possible even with completely independent trees. Thus, equality across trees would not be guaranteed. Instead, we first give a punctured key for the pre-tree. From this punctured key, the verifier can recover all the leaves of the pre-tree but one. Therefore, after hypercube folding, he learns exactly one node on the second level of each of the τ trees. Note that these nodes would normally be part of the punctured keys of the second level trees, so this technique allows us to remove one element from each of the second level punctured keys. Therefore, proceeding in this manner does not modify the overall size of the global punctured key. Indeed, we remove τ elements and replace them by the punctured key of the pre-tree, which contains τ nodes. To locate the missing position that is needed in the pre-tree, it suffices to assemble the high-order bits of the missing nodes of each second-level tree into a τ -bit number.

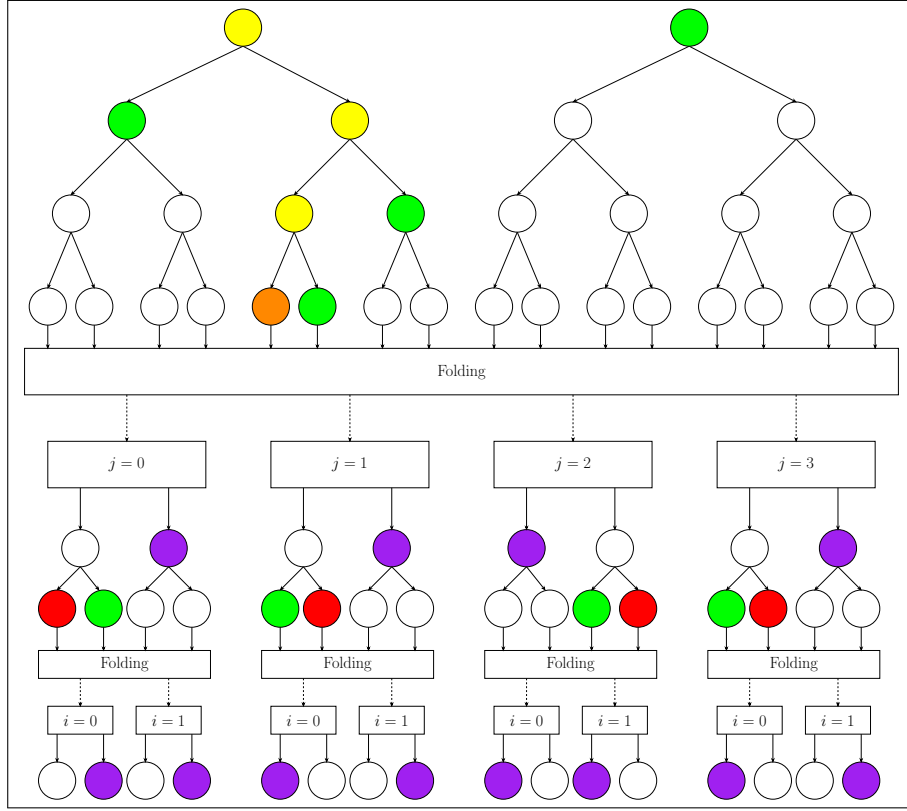


Fig. 2. Example of the punctured key for correlated GGM forest for $\tau = 4$ and $D = 2$. The challenges are indicated by red leaves, while the corresponding challenge in the pre-tree is colored orange. The yellow nodes correspond to the path from the root to the pre-tree challenge. The PPRF key contains all of the green nodes. The purple nodes can be recomputed with the **Folding** algorithm.

In Figure 2, we give an example of the punctured key for a correlated GGM forest which has $\tau = 4$ second layer trees with $N = 2^2$ leaves each. For this example, we assume that we receive the challenges $i_0^* = 0, i_1^* = 1, i_2^* = 3, i_3^* = 1$, indicated by the red leaves. First, we compute the corresponding challenge in the pre-tree as $i_p^* = \sum_{j=0}^{\tau-1} \lfloor i_j^* / 2^{D-1} \rfloor \cdot 2^j = 0 \cdot 2^0 + 0 \cdot 2^1 + 1 \cdot 2^2 + 0 \cdot 2^3 = 4$. Therefore, we need to compute the PPRF key for pre-tree at position $i_p^* = 4$, which is colored orange. To do that, we first consider the path from the root of the left half of the pre-tree to i_p^* (indicated by yellow nodes) and include the siblings of these nodes to the PPRF key $K_{i_p^*}$. We indicate those in green. With $K_{i_p^*}$, we can recompute all leaves of the pre-tree except at position $i_p^* = 4$. Using the hypercube folding, we can recompute one of the two hypercube shares for each index $j \in \{0, 1, 2, 3\}$. These nodes, colored purple, therefore do not have to be included in the PPRF key. Note that, since the leaf of the pre-tree at

position $i_p^* = 4$ is missing, we cannot compute the values of the siblings of these purple nodes. Lastly, we have to include the siblings of the remaining nodes on the paths to the red challenge leaves for each second layer tree to the PPRF key, which are also colored in green. Using the **Folding** algorithm again, we can recompute one hypercube share for each index $i \in [D]$, which are also colored purple. In particular, for our example in Figure 2, the hypercube sharings for $i = 0$ after folding have the form $(\vec{x}_j^{\llbracket 0 \rrbracket_0}, \vec{x}_j^{\llbracket 1 \rrbracket_0}) = ((\vec{x}_j^{\llbracket 0 \rrbracket} + \vec{x}_j^{\llbracket 2 \rrbracket}), (\vec{x}_j^{\llbracket 1 \rrbracket} + \vec{x}_j^{\llbracket 3 \rrbracket}))$, while for $i = 1$ they have the form $(\vec{x}_j^{\llbracket 0 \rrbracket_1}, \vec{x}_j^{\llbracket 1 \rrbracket_1}) = ((\vec{x}_j^{\llbracket 0 \rrbracket} + \vec{x}_j^{\llbracket 1 \rrbracket}), (\vec{x}_j^{\llbracket 2 \rrbracket} + \vec{x}_j^{\llbracket 3 \rrbracket}))$. Depending on which leaf is missing in tree $T_{\vec{x}_j}$, we can only compute one out of the two sums in each such sharing. As claimed, this PPRF key can be used to recompute a total of $8 = \tau \cdot D$ hypercube shares.

3.2 From Sharings to Vector Oblivious Linear Evaluation (VOLE)

In our use of the trees described in the above section, the prover always knows the complete trees, while the verifier only lacks one leaf of every tree (including the pre-tree). At the level of the sharings, this means that for every $m \in [\tau \cdot D]$, the verifier knows only one value $\vec{x}_m^{\llbracket b_m \rrbracket}$ (out of two) for a bit b_m of its choice, while the prover knows both.

It is equivalent and convenient to say that the prover creates an affine vectorial function:

$$f_m(b_m) = \vec{x}_m^{\llbracket 0 \rrbracket} \oplus b_m \cdot \vec{x}$$

and that the verifier chooses b_m and learns $f_m(b_m) = \vec{x}_m^{\llbracket b_m \rrbracket}$.

With this view in mind, we can now pick a series of random coefficients α_m in the large field $\mathbb{F}_{2^k}$ and consider the function:

$$\vec{F} = \sum_{m=0}^{\tau D-1} \alpha_m f_m(b_m) = \sum_{m=0}^{\tau D-1} \alpha_m \vec{x}_m^{\llbracket 0 \rrbracket} + \left[\sum_{m=0}^{\tau D-1} \alpha_m b_m \right] \cdot \vec{x}.$$

Remark that $\vec{F}$ is an affine function known to the prover, and that the verifier chooses the evaluation point $\Delta = \sum_{m=0}^{\tau D-1} \alpha_m b_m$ and learns the value $\vec{F}(\Delta)$ obtained by evaluating $\vec{F}$ at this point.

In principle, the coefficients α_m could be pre-specified as part of the scheme. However, to avoid casting suspicion on the choice of coefficients, we feel that it is preferable to have them chosen at random by the honest verifier.

3.3 A Simple but Inefficient Construction

As already mentioned, cGGM trees with internal nodes of size λ bits are limited and can only produce correlated values up to λ bits. Unfortunately, the secret key of the SBC problem contains a total of 2λ bits, namely $\vec{x}$ and $\vec{y}$. To illustrate our construction in a simple but inefficient way, let us use the construction of Section 3.1 twice, in order to independently get $\tau \cdot D$ sharings for both $\vec{x}$ and $\vec{y}$. We write them as:

$$\vec{x}_m^{[0]} \oplus \vec{x}_m^{[1]} = \vec{x} \quad \text{and} \quad \vec{y}_m^{[0]} \oplus \vec{y}_m^{[1]} = \vec{y} \quad \text{for each } m \in [\tau \cdot D]. \quad (2)$$

Proceeding as in Section 3.2, we use these two cGGM forests to define two vectorial functions $\vec{F}_x, \vec{F}_y : \mathbb{F}_{2^k} \rightarrow (\mathbb{F}_{2^k})^{n-2}$:

$$\begin{aligned} \vec{F}_x(\Delta) &= \vec{A}_{F_x} + \Delta \cdot \vec{x}, \\ \vec{F}_y(\Delta) &= \vec{A}_{F_y} + \Delta \cdot \vec{y}, \end{aligned}$$

whose constant coefficients are given by

$$\vec{A}_{F_x} = \vec{F}_x(0) = \sum_{m=0}^{\tau \cdot D - 1} \alpha_m \vec{x}_m^{[0]} \quad \text{and} \quad \vec{A}_{F_y} = \vec{F}_y(0) = \sum_{m=0}^{\tau \cdot D - 1} \alpha_m \vec{y}_m^{[0]}.$$

Note that, since we want to evaluate both functions at the same Δ , the missing tree leaves in both forests are located in the same manner.

Before learning the evaluation point Δ , the prover computes the polynomial Φ in Δ defined as:

$$\begin{aligned} \Phi(\Delta) &:= \left(\vec{u} \cdot \begin{pmatrix} \vec{F}_x(\Delta) \\ 1 \\ 0 \end{pmatrix} \right) \left(\vec{v} \cdot \begin{pmatrix} \vec{F}_y(\Delta) \\ 0 \\ 1 \end{pmatrix} \right) - \left(\vec{u} \cdot \begin{pmatrix} \vec{F}_y(\Delta) \\ 0 \\ 1 \end{pmatrix} \right) \left(\vec{v} \cdot \begin{pmatrix} \vec{F}_x(\Delta) \\ 1 \\ 0 \end{pmatrix} \right) \\ &= \left(\vec{u} \cdot \begin{pmatrix} \vec{A}_{F_x} \\ 1 \\ 0 \end{pmatrix} \right) \left(\vec{v} \cdot \begin{pmatrix} \vec{A}_{F_y} \\ 0 \\ 1 \end{pmatrix} \right) - \left(\vec{u} \cdot \begin{pmatrix} \vec{A}_{F_y} \\ 0 \\ 1 \end{pmatrix} \right) \left(\vec{v} \cdot \begin{pmatrix} \vec{A}_{F_x} \\ 1 \\ 0 \end{pmatrix} \right) \\ &\quad + \left[\left(\vec{v} \cdot \begin{pmatrix} \vec{A}_{F_y} \\ 0 \\ 1 \end{pmatrix} \right) \left(\vec{u} \cdot \begin{pmatrix} \vec{x} \\ 1 \\ 0 \end{pmatrix} \right) + \left(\vec{u} \cdot \begin{pmatrix} \vec{A}_{F_x} \\ 1 \\ 0 \end{pmatrix} \right) \left(\vec{v} \cdot \begin{pmatrix} \vec{y} \\ 0 \\ 1 \end{pmatrix} \right) \right. \\ &\quad \left. - \left(\vec{u} \cdot \begin{pmatrix} \vec{A}_{F_y} \\ 0 \\ 1 \end{pmatrix} \right) \left(\vec{v} \cdot \begin{pmatrix} \vec{x} \\ 1 \\ 0 \end{pmatrix} \right) - \left(\vec{v} \cdot \begin{pmatrix} \vec{A}_{F_x} \\ 1 \\ 0 \end{pmatrix} \right) \left(\vec{u} \cdot \begin{pmatrix} \vec{y} \\ 0 \\ 1 \end{pmatrix} \right) \right] \Delta \\ &\quad + \left[\left(\vec{u} \cdot \begin{pmatrix} \vec{x} \\ 1 \\ 0 \end{pmatrix} \right) \left(\vec{v} \cdot \begin{pmatrix} \vec{y} \\ 0 \\ 1 \end{pmatrix} \right) - \left(\vec{u} \cdot \begin{pmatrix} \vec{y} \\ 0 \\ 1 \end{pmatrix} \right) \left(\vec{v} \cdot \begin{pmatrix} \vec{x} \\ 1 \\ 0 \end{pmatrix} \right) \right] \Delta^2. \end{aligned}$$

Remark 2. Note that Φ is of the form

$$\Phi(\Delta) = A + B\Delta$$

if and only if $(\vec{x}', \vec{y}') \in \text{NSBC}[\vec{u}, \vec{v}]$ for $\vec{x}' = (\vec{x}, 1, 0)$, $\vec{y}' = (\vec{y}, 0, 1)$. The coefficients $A, B \in \mathbb{F}_{2^k}$ are given by

$$A = \left(\vec{u} \cdot \begin{pmatrix} \vec{A}_{F_x} \\ 1 \\ 0 \end{pmatrix} \right) \left(\vec{v} \cdot \begin{pmatrix} \vec{A}_{F_y} \\ 0 \\ 1 \end{pmatrix} \right) - \left(\vec{u} \cdot \begin{pmatrix} \vec{A}_{F_y} \\ 0 \\ 1 \end{pmatrix} \right) \left(\vec{v} \cdot \begin{pmatrix} \vec{A}_{F_x} \\ 1 \\ 0 \end{pmatrix} \right),$$

$$B = \begin{pmatrix} \vec{v} \cdot \begin{pmatrix} \vec{A}_{F_y} \\ 0 \\ 1 \end{pmatrix} \end{pmatrix} \begin{pmatrix} \vec{u} \cdot \begin{pmatrix} \vec{x} \\ 1 \\ 0 \end{pmatrix} \end{pmatrix} + \begin{pmatrix} \vec{u} \cdot \begin{pmatrix} \vec{A}_{F_x} \\ 1 \\ 0 \end{pmatrix} \end{pmatrix} \begin{pmatrix} \vec{v} \cdot \begin{pmatrix} \vec{y} \\ 0 \\ 1 \end{pmatrix} \end{pmatrix} \\ - \begin{pmatrix} \vec{u} \cdot \begin{pmatrix} \vec{A}_{F_y} \\ 0 \\ 1 \end{pmatrix} \end{pmatrix} \begin{pmatrix} \vec{v} \cdot \begin{pmatrix} \vec{x} \\ 1 \\ 0 \end{pmatrix} \end{pmatrix} - \begin{pmatrix} \vec{v} \cdot \begin{pmatrix} \vec{A}_{F_x} \\ 1 \\ 0 \end{pmatrix} \end{pmatrix} \begin{pmatrix} \vec{u} \cdot \begin{pmatrix} \vec{y} \\ 0 \\ 1 \end{pmatrix} \end{pmatrix}.$$

We use this property of Φ in our protocol.

The prover sends the coefficients $A, B \in \mathbb{F}_{2^k}$ to the verifier. The verifier responds by sending back random challenges $i_0^*, \dots, i_{\tau-1}^* \in [2^D]$, one for each tree. The prover returns the PPRF keys $K_{i_j^*}$ for the trees $T_{\vec{x}_j}$ and $T_{\vec{y}_j}$ (and the corresponding pre-trees) for each $j \in [\tau]$ to the verifier.

As explained in Section 3.2, this corresponds to the verifier choosing an evaluation point

$$\Delta^* := \sum_{m=0}^{\tau \cdot D - 1} b_m \alpha_m,$$

and obtaining the values $\vec{F}_x(\Delta^*)$ and $\vec{F}_y(\Delta^*)$.

Equipped with this knowledge, the verifier can compute $\Phi(\Delta^*)$ as

$$\Phi(\Delta^*) = \begin{pmatrix} \vec{u} \cdot \begin{pmatrix} \vec{F}_x(\Delta^*) \\ 1 \\ 0 \end{pmatrix} \end{pmatrix} \begin{pmatrix} \vec{v} \cdot \begin{pmatrix} \vec{F}_y(\Delta^*) \\ 0 \\ 1 \end{pmatrix} \end{pmatrix} - \begin{pmatrix} \vec{u} \cdot \begin{pmatrix} \vec{F}_y(\Delta^*) \\ 0 \\ 1 \end{pmatrix} \end{pmatrix} \begin{pmatrix} \vec{v} \cdot \begin{pmatrix} \vec{F}_x(\Delta^*) \\ 1 \\ 0 \end{pmatrix} \end{pmatrix}.$$

He then checks whether $\Phi(\Delta^*) = A + B\Delta^*$ and accepts or rejects accordingly.

Remark 3. The protocol accepts a wrong instance of vectors $(\vec{x}', \vec{y}') \notin \text{NSBC}[\vec{u}, \vec{v}]$ if Δ^* is a root of the polynomial

$$\Phi(\Delta) - B\Delta - A.$$

For $(\vec{x}', \vec{y}') \notin \text{NSBC}[\vec{u}, \vec{v}]$, $\Phi(\Delta)$ is a quadratic polynomial in Δ . Therefore, $\Phi(\Delta) - B\Delta - A$ has at most two roots. Since the evaluation point is chosen in a set of cardinality $2^{\tau D}$, the probability of getting a false positive is bounded by $2^{1-\tau D}$.

3.4 Consistency Checks

In Section 3.3, we use two independent cGGM forests. One to derive τD sharings of $\vec{x}$ and one to derive τD sharings of $\vec{y}$. Despite its simplicity, this approach has a very high cost, since the need for opening two forests using a puncturable PRF doubles the amount of data. In this section, we address the problem of implementing the same basic idea while using a single cGGM forest, based on $\vec{x}$. This leads to a significant decrease in the required communication cost for the scheme but requires additional techniques.

As mentioned in Section 3.1, we can use the cGGM forest provided to share $\vec{x}$ to create sharings of extra values, simply by applying a PRG to extend the size of individual leaves. However, this creates two problems. First, every tree induces sharings of random values, thus offsets are required to set the sharings to the desired value. As a consequence, each of the τ cGGM trees created to share $\vec{x}$ can be made to share $\vec{y}$ by providing a corresponding offset value. Since the communication costs for these offsets are much lower than for an extra cGGM forest, this is worthwhile in terms of signature size.

The second problem is more subtle and harder to fix. By providing an offset for $\vec{y}$, we can make sure that each of the τ cGGM trees provides D sharings of a value $\vec{y}_i$. However, nothing guarantees, from the verifier's viewpoint, that all these values are identical. This could be exploited by a cheating prover to boost the cheating probability. To remedy this, we need to add a consistency check that lets the verifier check that all the $\vec{y}_i$ for $i \in [\tau]$ are really equal. This idea of performing a (probabilistic) consistency check is known as the SoftSpokenOT technique [28]. In our case, it can be somewhat simplified, and we now describe our version in detail.

For the purpose of realizing the equality testing, we extend each leaf in the cGGM forest created to share $\vec{x}$ in order to produce additional sharings of two additional elements: the second secret value $\vec{y}$ and an extra element z from $\mathbb{F}_{2^k}$. Since $\vec{y}$ is fixed, one offset is required for each of the τ trees. For z , since a random element is requested, it is tempting to assume that no offsets are needed. However, to prove equality between sharings of $\vec{y}$, we also need the sharings of z to be equal across the trees. As a consequence, we can only skip the offset corresponding to z for the first of the cGGM trees in the family.

More precisely, from each leaf $\vec{x}_j^{[i]} \in (\mathbb{F}_2)^{n-2}$, the prover derives the shares $\vec{y}_j^{[i]} \in (\mathbb{F}_2)^{n-2}$ and $z_j^{[i]} \in \mathbb{F}_{2^k}$ for each $(i, j) \in [2^D] \times [\tau]$, using the random oracle $\mathcal{H}$. To commit to the family of τ subtrees, the prover sends the offset values $\delta_{\vec{y}_j} \in (\mathbb{F}_2)^{n-2}$ and $\delta_{z_j} \in \mathbb{F}_{2^k}$ such that

$$\vec{y}_j = \sum_{i=0}^{2^D-1} \vec{y}_j^{[i]} + \delta_{\vec{y}_j} \text{ and } z_j = \sum_{i=0}^{2^D-1} z_j^{[i]} + \delta_{z_j} \text{ for } j \in [\tau], \quad (3)$$

to the verifier. Note that we set δ_{z_0} to 0 and do not send this value.

After the prover has committed to the cGGM trees, the verifier computes a random vector $\vec{\mu} \in (\mathbb{F}_{2^k})^{n-2}$ and sends it to the prover. The goal of $\vec{\mu}$ is to provide an efficient check that the pairs $(\vec{y}_j, z_j)$ are equal among the rounds. Since $\vec{\mu}$ is selected randomly and not under the control of the prover, it suffices to check that $z_j + \vec{\mu} \cdot \vec{y}_j$ is constant across the rounds.

Remark 4. Note that the goal of z is to act as a one-time-pad and hide the value $\vec{\mu} \cdot \vec{y}$ which would, if revealed, leak the secret $\vec{y}$.

This equality check relies on the following lemma.

Lemma 1. Let $\vec{y}, \vec{y}' \in (\mathbb{F}_2)^{n-2}$ and $z, z' \in \mathbb{F}_{2^k}$ with $(\vec{y}, z) \neq (\vec{y}', z')$ and let $\vec{\mu} \xleftarrow{\$} (\mathbb{F}_{2^k})^{n-2}$. Then $\Pr[z + \vec{\mu} \cdot \vec{y} = z' + \vec{\mu} \cdot \vec{y}'] \leq 2^{-k}$, where the probability is on the choice of $\vec{\mu}$.

Proof. Since $(\vec{y}, z) \neq (\vec{y}', z')$, we have:

$$(\vec{y} \oplus \vec{y}', z \oplus z') \neq (\vec{0}, 0).$$

To analyze the probability $\Pr[z + \vec{\mu} \cdot \vec{y} = z' + \vec{\mu} \cdot \vec{y}']$, we consider two cases:

1. Assume $\vec{y} \oplus \vec{y}' = \delta_{\vec{y}} \neq \vec{0}$. In this situation, the probabilistic event occurs when $\delta_{\vec{y}} \cdot \vec{\mu} = z \oplus z'$. Equivalently, $\vec{\mu}$ belongs to a given affine hyperplane orthogonal to $\delta_{\vec{y}}$, which occurs with (conditional) probability 2^{-k} .
2. Assume $\vec{y} = \vec{y}'$ and $z \neq z'$, thus $z + \vec{\mu} \cdot \vec{y} \neq z' + \vec{\mu} \cdot \vec{y}'$. In this case, the (conditional) probability is 0.

The statement follows by combining both cases. $\square$

We now go back to the construction of the VOLE for $\vec{y}$ and z . A priori, the verifier is not sure that $\vec{y}$ and z are equal in all trees, and the construction from Section 3.2 cannot be used without precaution. However, all the binary sharings derived from an individual tree correspond to equal values $(\vec{y}_j, z_j)$, where $j \in [\tau]$ denotes the index of the tree in its family. Thus, the prover can compute a VOLE instance (with smaller evaluation domain) for $(\vec{y}_j, z_j)$ for each $j \in [\tau]$. More precisely, after folding, each tree provides D binary sharings:

$$\left(\vec{y}_j^{\llbracket 0 \rrbracket_i}, z_j^{\llbracket 0 \rrbracket_i} \right) \oplus \left(\vec{y}_j^{\llbracket 1 \rrbracket_i}, z_j^{\llbracket 1 \rrbracket_i} \right) \oplus (\delta_{\vec{y}_j}, \delta_{z_j}) = (\vec{y}_j, z_j).$$

As in Section 3.3, for each value $(i, j) \in [D] \times [\tau]$ the prover defines a linear polynomial:

$$f_{i,j}(b) = \left(\vec{y}_j^{\llbracket b \rrbracket_i}, z_j^{\llbracket 0 \rrbracket_i} \right) + b \cdot (\vec{y}_j, z_j).$$

We have grouped $\vec{y}_j$ and z_j in the above polynomial for compactness. However, by abuse of notation and for the sake of convenience, we might refer to it as a polynomial in $\vec{y}_j$ or z_j alone. We continue as in Section 3.3, and combine the D polynomials in $\vec{y}_j$ and z_j together. This defines a polynomial:

$$\vec{F}_{y,z}^{(j)}(\Delta_j) = \sum_{i=0}^{D-1} \alpha_{D-j+i} f_{i,j}(b_{i,j}) = \sum_{i=0}^{D-1} \alpha_{D-j+i} \left(\vec{y}_j^{\llbracket b \rrbracket_i}, z_j^{\llbracket 0 \rrbracket_i} \right) + \Delta_j \cdot (\vec{y}_j, z_j),$$

where $\Delta_j = \sum_{i=0}^{D-1} \alpha_{D-j+i} b_{i,j}$. As before, we can write this as

$$\vec{F}_{y,z}^{(j)}(\Delta_j) = \vec{A}_{F_{y,z}}^{(j)} + \Delta_j \cdot (\vec{y}_j, z_j), \tag{4}$$

where the coefficient $\vec{A}_{F_{y,z}}^{(j)} = \vec{F}_{y,z}^{(j)}(0)$ is given by the evaluation at 0.

To implement the test provided by Lemma 1, we combine $\vec{\mu}$ with Equation (4) and find:

$$\begin{aligned}\vec{F}_{y,z}^{(j)}(\Delta_j) \cdot (\vec{\mu}, 1) &= \vec{A}_{F_{y,z}}^{(j)} \cdot (\vec{\mu}, 1) + \Delta_j(\vec{y}_j, z_j) \cdot (\vec{\mu}, 1) \\ &= \vec{A}_{F_{y,z}}^{(j)} \cdot (\vec{\mu}, 1) + \Delta_j[z_j + \vec{y}_j \cdot \vec{\mu}].\end{aligned}$$

To understand the goal of the protocol, let us first **assume** that $\vec{y}_j$ and z_j are **equal across trees**. Then the coefficient in front of Δ_j is the same for all values of j . As a consequence, we ask that the prover computes and transmit this coefficient $b = z + \vec{y} \cdot \vec{\mu} \in \mathbb{F}_{2^k}$, where $\vec{y}$ and z denote the (putative) common values across trees. The prover also computes and transmits the coefficients $a^{(j)} = \vec{A}_{F_{y,z}}^{(j)} \cdot (\vec{\mu}, 1) \in \mathbb{F}_{2^k}$ for all $j \in [\tau]$.

As usual, the verifier sends the challenges $i_j^* \in [2^D]$ for each subtree T_j and obtains the corresponding punctured key from the prover, thus learning the evaluations $\vec{F}_{y,z}^{(j)}(\Delta_j^*)$ for each of the τ trees. Let us remark that if $\vec{y}_j$ and z_j are equal across trees, then we can add these evaluations and obtain:

$$\vec{F}_{y,z}(\Delta^*) = \sum_{j=0}^{\tau-1} \vec{F}_{y,z}^{(j)}(\Delta_j^*) = \vec{F}_{y,z}(0) + \Delta^*(\vec{y}, z), \text{ where } \Delta^* = \sum_{j=0}^{\tau-1} \Delta_j^*.$$

In particular, the verifier learns $\vec{F}_y(\Delta^*)$ and also $\vec{F}_x(\Delta^*)$ (from the correlated part of the family of trees).

Thus, as in the simplified version of Section 3.3, the verifier computes

$$\Phi(\Delta^*) = \left(\vec{u} \cdot \begin{pmatrix} \vec{F}_x(\Delta^*) \\ 1 \\ 0 \end{pmatrix} \right) \left(\vec{v} \cdot \begin{pmatrix} \vec{F}_y(\Delta^*) \\ 0 \\ 1 \end{pmatrix} \right) - \left(\vec{u} \cdot \begin{pmatrix} \vec{F}_y(\Delta^*) \\ 0 \\ 1 \end{pmatrix} \right) \left(\vec{v} \cdot \begin{pmatrix} \vec{F}_x(\Delta^*) \\ 1 \\ 0 \end{pmatrix} \right),$$

checks if it is equal to $A + B\Delta^*$ and accepts or rejects accordingly.

If we **no longer assume equality across trees**, the verifier has to perform a little extra work. Namely, after obtaining each partial evaluation $\vec{F}_{y,z}^{(j)}(\Delta_j^*)$, he computes $\vec{F}_{y,z}^{(j)}(\Delta_j^*) \cdot (\vec{\mu}, 1)$ and checks that the value is equal to $a^{(j)} + b\Delta_j^*$. If any of these checks fail, the verifier rejects. If all checks succeed, he continues with the computation described in the case of equality, computing and checking $\Phi(\Delta^*)$. The following theorem states that this provides a sound protocol.

Theorem 1. *The soundness error of the NSBC identification scheme described above using $N = 2^D$ parties and τ rounds is bounded by $\tau^2 2^{-k} + 2^{1-\tau \cdot D}$.*

Proof. Consider a prover who does not share the same value for $(\vec{y}, z)$ in every round. The values $(\vec{y}_i, z_i)$ can be partitioned into at most τ distinct subsets. In particular, we have that

$$\{(\vec{y}_0, z_0), \dots, (\vec{y}_{\tau-1}, z_{\tau-1})\} = S^{(0)} \dot{\cup} \dots \dot{\cup} S^{(m-1)}, \quad (5)$$

where $m \leq \tau$. For any fixed index $j \in [m]$, we have that $(\vec{y}_{i_0}, z_{i_0}) = (\vec{y}_{i_1}, z_{i_1})$ for all $(\vec{y}_{i_0}, z_{i_0}), (\vec{y}_{i_1}, z_{i_1}) \in S^{(j)}$ by definition. Moreover, for any $j_0, j_1 \in [m]$ with $j_0 \neq j_1$, $S^{(j_0)} \cap S^{(j_1)} = \emptyset$.

From Lemma 1 and the union bound, we know that

$$\Pr[\exists i \neq j | (\vec{y}_i, z_i) \in S^{(i)}, (\vec{y}_j, z_j) \in S^{(j)} \text{ and } z_i + \vec{y}_i \cdot \vec{\mu} = z_j + \vec{y}_j \cdot \vec{\mu}] \leq \tau^2 2^{-k}.$$

This *bad event* provides the term $\tau^2 2^{-k}$ in the soundness error.

We now assume that the bad event has not occurred and without loss of generality, we consider that the reference value is $(\vec{y}_0, z_0) \in S^{(0)}$ and that during the protocol the prover sends the value $b = z_0 + \vec{y}_0 \cdot \vec{\mu}$. Since we excluded the above bad event, we know that $z_j + \vec{y}_j \cdot \vec{\mu} \neq b$ for all $(\vec{y}_j, z_j) \in S^{(i)}$, where $i \in \{1, \dots, m-1\}$.

Consider a subtree T_j for $j \in [\tau]$ with $(a^{(j)}, b^{(j)}) \neq (a^{(j)}, b)$. For this index, we see that $a^{(j)} + b\Delta_j = z_j + \vec{\mu} \cdot \vec{F}_y^{(j)}(\Delta_j)$ for at most one value Δ_j . Indeed, the affine function $a^{(j)} + b\Delta_j - z_j - \vec{\mu} \cdot \vec{F}_y^{(j)}(\Delta_j)$ is non zero and thus has at most one root.

In order for the verifier to validate the equality testing for this index j , a cheating prover needs to guess the evaluation point Δ_j^* in advance and provide a corresponding $a^{(j)}$. We denote the subset of $[\tau]$ containing the indices j with $z_j + \vec{y}_j \cdot \vec{\mu} \neq b$ as I_{bad} . In particular,

$$I_{\text{bad}} := \{j \in [\tau] | z_j + \vec{y}_j \cdot \vec{\mu} \neq b\}.$$

Since each set of possible evaluation points for the partial VOLE evaluation of any single tree contains 2^D points, the probability for the prover to correctly guess all evaluation points for I_{bad} is:

$$\Pr[\text{Prover guesses all } \Delta_j \text{ for } j \in I_{\text{bad}}] = 2^{-D \cdot |I_{\text{bad}}|}.$$

If the prover guesses correctly, we can treat all the Δ_j for $j \in I_{\text{bad}}$ as constant values. As a consequence, the linear polynomials that are constructed in this situation only depend on the remaining indices $I_{\text{good}} := [\tau] \setminus I_{\text{bad}}$. More precisely, everything can be expressed in terms of the variable:

$$\Delta_g = \sum_{j \in I_{\text{good}}} \Delta_j.$$

In particular, we can view Φ as a polynomial in Δ_g . Furthermore, we can see that $\Phi(\Delta_g)$ is an affine polynomial if and only if $(\vec{x}', \vec{y}_g') \in \text{NSBC}[\vec{u}, \vec{v}]$ for $\vec{x}' = (\vec{x}, 1, 0)$, $\vec{y}_g' = (\vec{y}_g, 0, 1)$.

As a consequence, for a cheating prover that does not know a solution to the NSBC instance, we know that $\Phi(\Delta_g)$ is a non-zero polynomial of degree 2. Thus, the linearity check on Φ can only work at two exceptional points (coming from a quadratic equation). Thus, the probability that this test succeeds for a cheating prover is upper bounded by $\frac{2}{2^{D \cdot |I_{\text{good}}|}} = 2^{1-D \cdot |I_{\text{good}}|}$. Since $\tau = |I_{\text{good}}| + |I_{\text{bad}}|$ and

since the equality test for I_{bad} and the linearity test for I_{good} are independent, the overall success probability of both is upper bounded by:

$$2^{-D \cdot |I_{\text{bad}}|} \cdot 2^{1-D \cdot |I_{\text{good}}|} = 2^{1-\tau \cdot D}.$$

Adding this to the probability of the bad event described earlier, we conclude that the soundness error of the identification scheme is bounded by:

$$\tau^2 2^{-k} + 2^{1-\tau \cdot D}.$$

□

4 Detailed Description of the Identification Scheme

For completeness, we now provide a detailed description of our five-round identification protocol between the prover $\mathcal{P}$ and the honest verifier $\mathcal{V}$. This protocol uses six algorithmic sub-routines, executed in alternation by the prover and the verifier. These algorithms are: Commitment, First Challenge, First Response, Second Challenge, Second Response, and Verification.

The main cryptographic tool that we use is the correlated GGM forest from Section 3.1. In this section, we base the construction on a random oracle $\mathcal{H}_2$ with output size λ . Following the state of the art, we use a **salt** of size 2λ bits and also do full domain separation, providing a tree number and a node position to derive the left child of each node using $\mathcal{H}_2$. In particular, the left child of a node in tree j (where $j = \text{pre}$ or $j \in [\tau]$) at position $\text{pos} = (\text{Level}, \text{index})$ is derived as

$$\mathcal{H}_2(\text{salt} \parallel j \parallel \text{pos} \parallel \text{node value}).$$

The right child is obtained by XORing the left child and the parent. At the level of the leaf, we also use a random oracle (with proper domain separation) to derive additional shares for $\vec{y}$ and z as well as a commitment value. To prevent the attack from [26], we use explicit commitments to the tree of size 2λ bits. However, we propose a new MinMax technique to commit to the cGGM forest in Section 6.1 to reduce the signature size compared to the basic version. With our choice of security parameters, we need to create $5\lambda + 1$ additional bits for each leaf. The commitment algorithm is shown in Figure 3. Figures 4 and 5 present the first challenge and response, while Figures 6 and 7 detail the second challenge and response. The verification algorithm is given in Figure 8.

5 Signature Scheme

We can now proceed to transform the Honest Verifier Zero Knowledge (HVZK) protocol from Section 3 into a signature scheme by using the Fiat-Shamir transform [18] to obtain the desired security level of λ bits.

The public key is given by the two vectors $\vec{u}, \vec{v} \in (\mathbb{F}_{q^k})^n$, while the secret key consists of $\vec{x}, \vec{y} \in (\mathbb{F}_q)^{n-2}$ with $(\vec{x}', \vec{y}') \in \text{NSBC}[\vec{u}, \vec{v}]$ for $\vec{x}' := (\vec{x}, 1, 0)$ and $\vec{y}' := (\vec{y}, 0, 1)$.

Standard commitment

1. Sample $R_{\vec{x}} \xleftarrow{\$} (\mathbb{F}_2)^{n-2}$ and a random salt $\xleftarrow{\$} \{0,1\}^{2\lambda}$ for the cGGM trees derivation
2. Compute pre-tree: $(\vec{x}^{[0]}, \dots, \vec{x}^{[2^\tau-1]}) \leftarrow \text{cGGM}_{\text{salt}}(R_{\vec{x}}, R_{\vec{x}} \oplus \vec{x}, 2^\tau)$
3. Fold pre-tree: $(\vec{x}^{[0]_j}, \vec{x}^{[1]_j})_{j \in [\tau]} \leftarrow \text{Folding}(\vec{x}^{[0]}, \dots, \vec{x}^{[2^\tau-1]})$
4. For $j \in [\tau]$:
 - Get Tree T_j : $(\vec{x}_j^{[0]}, \dots, \vec{x}_j^{[2^D-1]}) \leftarrow \text{cGGM}_{\text{salt}}(\vec{x}^{[0]_j}, \vec{x}^{[1]_j}, 2^D)$
 - Expand T_j and get $(\vec{y}_j^{[0]}, \dots, \vec{y}_j^{[2^D-1]})$, $(z_j^{[0]}, \dots, z_j^{[2^D-1]})$ and $(\text{com}_j^{[0]}, \dots, \text{com}_j^{[2^D-1]})$
 - Compute $X_j \leftarrow \bigoplus_{i=0}^{2^D-1} \text{com}_j^{[i]}$
 - Fold Tree T_j :
 - $(\vec{x}_j^{[0]_i}, \vec{x}_j^{[1]_i})_{i \in [D]} \leftarrow \text{Folding}(\vec{x}_j^{[0]}, \dots, \vec{x}_j^{[2^D-1]})$
 - $(\vec{y}_j^{[0]_i}, \vec{y}_j^{[1]_i})_{i \in [D]} \leftarrow \text{Folding}(\vec{y}_j^{[0]}, \dots, \vec{y}_j^{[2^D-1]})$
 - $(z_j^{[0]_i}, z_j^{[1]_i})_{i \in [D]} \leftarrow \text{Folding}(z_j^{[0]}, \dots, z_j^{[2^D-1]})$
 - $(\text{com}_j^{[0]_i}, \text{com}_j^{[1]_i})_{i \in [D]} \leftarrow \text{Folding}(\text{com}_j^{[0]}, \dots, \text{com}_j^{[2^D-1]})$
 - Let $z \leftarrow z_0^{[0]_0} \oplus z_0^{[1]_0}$
 - Compute offsets:
 - $\delta_{\vec{y}_j} \leftarrow \vec{y} \oplus \vec{y}_j^{[0]_0} \oplus \vec{y}_j^{[1]_0}$
 - $\delta_{z_j} \leftarrow z \oplus z_j^{[0]_0} \oplus z_j^{[1]_0}$
5. Compute $h_{\text{com}} \leftarrow \mathcal{H}_3\left(\text{salt}, (\text{com}_j^{[0]_i}, \text{com}_j^{[1]_i})_{(i,j) \in [D] \times [\tau]}, (X_j)_{j \in [\tau]}\right)$
6. Send $h_{\text{com}}, (\delta_{\vec{y}_j})_{j \in [\tau]}, (\delta_{z_j})_{j \in [\tau]}, (X_j)_{j \in [\tau]}$ and salt to the verifier

Fig. 3. Commitment algorithm of the identification protocol for the NSBC problem

5.1 Removing the Interactivity of the Protocol

In the signature scheme, all algorithms of the prover and verifier but the final verification are executed by the signer. Only the final verification is performed by the verifier.

Using the five-round Fiat-Shamir transform, we simply replace the first and second challenge by hash values based on two random oracles $\mathcal{H}_0, \mathcal{H}_1$ with output size 2λ bits each. Given a message $\text{msg} \in \{0,1\}^*$, we derive the first challenge as:

$$h_0 \leftarrow \mathcal{H}_0\left(\text{msg}, h_{\text{com}}, \text{salt}, (\delta_{\vec{y}_j}, \delta_{z_j})_{j \in [\tau]}, (X_j)_{j \in [\tau]}\right)$$

and expand it into $\vec{\mu}$ and $(\alpha_i)_{i \in [\tau \cdot D]}$ using a PRG.³

³ We also assume that $\mathcal{H}_0$ takes additional context, like for example the public key pk , as input. By doing that, any forgery attempt is tied to a specific user. To simplify the notation, we do not specifically write this in every input.

Challenge 1

1. Sample $\vec{\mu} \xleftarrow{\$} (\mathbb{F}_{2^k})^{n-2}$
2. For $m \in [\tau \cdot D]$: Sample $\alpha_m \xleftarrow{\$} \mathbb{F}_{2^k}$
3. Send $\vec{\mu}, (\alpha_i)_{i \in [\tau \cdot D]}$ to the prover

Fig. 4. First Challenge algorithm of the identification protocol for the NSBC problem

Response 1

1. $\vec{A}_{F_x} \leftarrow \sum_{j=0}^{\tau-1} \sum_{i=j \cdot D}^{(j+1) \cdot D-1} \vec{x}_j^{[0]i} \alpha_i$
2. $\vec{A}_{F_y} \leftarrow \sum_{j=0}^{\tau-1} \sum_{i=j \cdot D}^{(j+1) \cdot D-1} \vec{y}_j^{[0]i} \alpha_i$
3. $A \leftarrow \left(\vec{u} \cdot \begin{pmatrix} \vec{A}_{F_x} \\ 1 \\ 0 \end{pmatrix} \right) \left(\vec{v} \cdot \begin{pmatrix} \vec{A}_{F_y} \\ 0 \\ 1 \end{pmatrix} \right) - \left(\vec{u} \cdot \begin{pmatrix} \vec{A}_{F_y} \\ 0 \\ 1 \end{pmatrix} \right) \left(\vec{v} \cdot \begin{pmatrix} \vec{A}_{F_x} \\ 1 \\ 0 \end{pmatrix} \right)$
4. $B \leftarrow \left(\vec{v} \cdot \begin{pmatrix} \vec{A}_{F_y} \\ 0 \\ 1 \end{pmatrix} \right) \left(\vec{u} \cdot \begin{pmatrix} \vec{x} \\ 1 \\ 0 \end{pmatrix} \right) + \left(\vec{u} \cdot \begin{pmatrix} \vec{A}_{F_x} \\ 1 \\ 0 \end{pmatrix} \right) \left(\vec{v} \cdot \begin{pmatrix} \vec{y} \\ 0 \\ 1 \end{pmatrix} \right) - \left(\vec{u} \cdot \begin{pmatrix} \vec{A}_{F_y} \\ 0 \\ 1 \end{pmatrix} \right) \left(\vec{v} \cdot \begin{pmatrix} \vec{x} \\ 1 \\ 0 \end{pmatrix} \right) - \left(\vec{v} \cdot \begin{pmatrix} \vec{A}_{F_x} \\ 1 \\ 0 \end{pmatrix} \right) \left(\vec{u} \cdot \begin{pmatrix} \vec{y} \\ 0 \\ 1 \end{pmatrix} \right)$
5. Compute $b \leftarrow z + \vec{y} \cdot \vec{\mu}$
6. For $j \in [\tau]$:
 - Compute $\vec{A}_{F_y}^{(j)} \leftarrow \sum_{i=j \cdot D}^{(j+1) \cdot D-1} \vec{y}_j^{[0]i} \alpha_i$ and $\vec{A}_{F_z}^{(j)} \leftarrow \sum_{i=j \cdot D}^{(j+1) \cdot D-1} z_j^{[0]i} \alpha_i$
 - Compute $a^{(j)} \leftarrow \vec{A}_{F_z}^{(j)} + \vec{A}_{F_y}^{(j)} \cdot \vec{\mu}$
7. Send $A, B, (a^{(j)})_{j \in [\tau]}$ and b to the verifier

Fig. 5. First Response algorithm of the identification protocol for the NSBC problem

Note that neither h_0 nor the derived $\vec{\mu}$ and $(\alpha_i)_{i \in [\tau \cdot D]}$ need to be sent as part of the signature since they can be recomputed from the offsets $\delta_{\vec{y}_j}$ and δ_{z_j} using a pseudorandom generator PRG_0 .

To derive the second challenge, we continue hashing, including now A , B , $(a^{(j)})_{j \in [\tau]}$ and b into a hash h_1 from which we obtain the second challenge $(i_j^*)_{j \in [\tau]}$ using PRG_1 . This also permits additional reduction of the signature size. Indeed, since A and $(a^{(j)})_{j \in [\tau]}$ are recomputed during the verification phase, they do not need to be sent. Instead, sending (and verifying) the hash h_1 is enough. We provide pseudocode of the algorithms for the SBC signature scheme below. The Signing algorithm is given in Figure 9, while the Verification algorithm is given in Figure 10.

Challenge 2

1. Sample $(i_j^*)_{j \in [\tau]} \xleftarrow{\$} [2^D]^\tau$ and send it to the prover

Fig. 6. Second Challenge algorithm of the identification protocol for the NSBC problem

Response 2

1. Compute the pre-challenge $i_p^* \leftarrow \sum_{j=0}^{\tau-1} \lfloor i_j^*/2^{D-1} \rfloor 2^j$
2. Compute the pre-tree PPRF key $K_{i_p^*}$
3. Compute the (truncated) PPRF key $K_{i_0^*, \dots, i_{\tau-1}^*}$ of the second layer trees
4. Send $(K_{i_p^*}, K_{i_0^*, \dots, i_{\tau-1}^*})$ to the verifier

Fig. 7. Second Response algorithm of the identification protocol for the NSBC problem

5.2 Signature Size

As mentioned in [21], we need the bitsize of the elements of $\mathbb{F}_{q^k}$ to be at least 2λ to avoid generic attacks based on collision finding. Therefore, since $n \approx \frac{k}{2}$, the bitsize of the elements of $(\mathbb{F}_q)^{n-2}$ is λ . The signature consists of the following elements:

- One hash value corresponding to the global commitment of size 2λ .
- One salt of size 2λ .
- The punctured key of the correlated GGM forest of size $\lambda\tau \log_2(N)$.
- The coefficient $B \in \mathbb{F}_{q^k}$.
- The coefficient $b = z_0 + \vec{y}_0 \cdot \vec{\mu} \in \mathbb{F}_{q^k}$.
- The offset values $\delta_{\vec{y}_0}, \dots, \delta_{\vec{y}_{\tau-1}} \in (\mathbb{F}_q)^{n-2}$ and $\delta_{z_1}, \dots, \delta_{z_{\tau-1}} \in \mathbb{F}_{q^k}$.
- **Explicit tree commitment**⁴: the XORs $(X_j)_{j \in [\tau]}$ of the commitment values of each tree of size $\tau \cdot 2\lambda$.

The total communication cost in bits of the protocol is therefore

$$\begin{aligned} \text{size} &\geq \lambda\tau \log_2(N) + \tau\lambda + 2\tau\lambda + (\tau-1)2\lambda + 2\lambda + 2\lambda + 2\lambda + 2\lambda \\ &= \lambda\tau \log_2(N) + 5\tau\lambda + 6\lambda \\ &\approx \lambda^2 + 5\tau\lambda + 7\lambda. \end{aligned}$$

The final approximate size depends on the exact value of $\tau \log_2(N)$. We need $\tau \log_2(N) > \lambda$ and take $\tau \log_2(N) \approx \lambda + 1$, as far as factorization of numbers close to $\lambda + 1$ permits.

From Theorem 1, we know that the total soundness error of the identification protocol we use is bounded by

$$\tau^2 2^{-k} + 2^{1-\tau \cdot D}. \quad (6)$$

⁴ In Section 6.1, we introduce a technique to remove this explicit commitment to shorten the signature.

Verification

1. Recompute, expand and fold all trees (with one missing leaf each) using $(K_{i_p^*}, K_{i_0^*, \dots, i_{\tau-1}^*})$ and **salt**
2. For $(i, j) \in [D] \times [\tau]$, let $b_{i,j}$ denote the known position of the respective hypercube binary sharing
3. For $(i, j) \in [D] \times [\tau]$: Compute $\text{com}_j^{\llbracket 1-b_{i,j} \rrbracket i} \leftarrow X_j \oplus \text{com}_j^{\llbracket b_{i,j} \rrbracket i}$
4. $\delta_{z_0} \leftarrow 0$
5. For $j \in [\tau]$:
 - Compute $\Delta_j^* \leftarrow \sum_{i=j \cdot D}^{(j+1) \cdot D-1} b_{i,j} \alpha_i$
 - Compute $\vec{F}_y^{(j)}(\Delta_j^*) \leftarrow \sum_{i=j \cdot D}^{(j+1) \cdot D-1} \alpha_i \left(\vec{y}_j^{\llbracket b_{i,j} \rrbracket i} + b_{i,j} \delta_{\vec{y}_j} \right)$
 - Compute $Z^{(j)}(\Delta_j^*) \leftarrow \sum_{i=j \cdot D}^{(j+1) \cdot D-1} \alpha_i \left(z_j^{\llbracket b_{i,j} \rrbracket i} + b_{i,j} \delta_{z_j} \right)$
 - $a^{(j)'} \leftarrow Z^{(j)}(\Delta_j^*) \oplus \vec{F}_y^{(j)}(\Delta_j^*) \cdot \vec{\mu} \oplus b \Delta_j^*$
6. $\Delta^* \leftarrow \sum_{j=0}^{\tau-1} \Delta_j^*$
7. $\vec{F}_x(\Delta^*) \leftarrow \sum_{m=0}^{\tau \cdot D-1} \vec{x}_m^{\llbracket b_m \rrbracket} \alpha_m$
8. $\vec{F}_y(\Delta^*) \leftarrow \sum_{j=0}^{\tau-1} \vec{F}_y^{(j)}(\Delta_j^*)$
9. Compute $\Phi(\Delta^*)$ as

$$\left(\vec{u} \cdot \begin{pmatrix} \vec{F}_x(\Delta^*) \\ 1 \\ 0 \end{pmatrix} \right) \left(\vec{v} \cdot \begin{pmatrix} \vec{F}_y(\Delta^*) \\ 0 \\ 1 \end{pmatrix} \right) - \left(\vec{u} \cdot \begin{pmatrix} \vec{F}_y(\Delta^*) \\ 0 \\ 1 \end{pmatrix} \right) \left(\vec{v} \cdot \begin{pmatrix} \vec{F}_x(\Delta^*) \\ 1 \\ 0 \end{pmatrix} \right)$$
10. $A' \leftarrow \Phi(\Delta^*) - B \Delta^*$
11. Compute $h'_{\text{com}} \leftarrow \mathcal{H}_3 \left(\text{salt}, \left(\text{com}_j^{\llbracket 0 \rrbracket i}, \text{com}_j^{\llbracket 1 \rrbracket i} \right)_{(i,j) \in [D] \times [\tau]}, (X_j)_{j \in [\tau]} \right)$
12. If $h'_{\text{com}} = h_{\text{com}}$, $A = A'$ and $(a^{(j)})_{j \in [\tau]} = (a^{(j)'})_{j \in [\tau]}$ output **ACCEPT**, otherwise output **REJECT**

Fig. 8. Verification algorithm of the identification protocol for the NSBC problem

In general, for a signature scheme based on the Fiat-Shamir transformation of a 5 round protocol, the cost of forgery is given by the Kales and Zaverucha formula, based on their attack from [24]:

$$\text{cost} = \min_{\tau_1, \tau_2: \tau_1 + \tau_2 = \tau} \left\{ \frac{1}{\sum_{i=\tau_1}^{\tau} \binom{\tau}{i} p^i (1-p)^{\tau-i}} + N^{\tau_2} \right\},$$

where p denotes the false positive probability of the underlying identification scheme.

In our case, the false positive probability, in the sense of Kales and Zaverucha, is $p = \tau 2^{-k}$ from Lemma 1. Since this is extremely small compared to the desired security level, the Kales and Zaverucha attack does not apply in our case. In fact, our proof of Theorem 1 uses a union bound argument to prevent any false positive from happening, and the same proof directly shows the security level of our signature scheme in the random oracle model.

Consider the security level of $\lambda = 128$ bits and the parameters $q = 2, k = 257$ and $n = 130$. For these parameters, the values for D and τ , including the

- Inputs:** Secret key $\text{sk} = (\vec{x}, \vec{y})$, public key $\text{pk} = (\vec{u}, \vec{v})$ and a message $\text{msg} \in \{0, 1\}^*$
1. Sample $R_{\vec{x}} \xleftarrow{\$} (\mathbb{F}_2)^{n-2}$ and $\text{salt} \xleftarrow{\$} \{0, 1\}^{2\lambda}$ for the cGGM tree derivation
 2. Compute pre-tree: $(\vec{x}^{[0]}, \dots, \vec{x}^{[2^\tau-1]}) \leftarrow \text{cGGM}_{\text{salt}}(R_{\vec{x}}, R_{\vec{x}} \oplus \vec{x}, 2^\tau)$
 3. Fold pre-tree: $(\vec{x}^{[0]_j}, \vec{x}^{[1]_j})_{j \in [\tau]} \leftarrow \text{Folding}(\vec{x}^{[0]}, \dots, \vec{x}^{[2^\tau-1]})$
 4. For $j \in [\tau]$:
 - Get Tree T_j : $(\vec{x}_j^{[0]}, \dots, \vec{x}_j^{[2^D-1]}) \leftarrow \text{cGGM}_{\text{salt}}(\vec{x}^{[0]_j}, \vec{x}^{[1]_j}, 2^D)$
 - Expand T_j and get $(\vec{y}_j^{[0]}, \dots, \vec{y}_j^{[2^D-1]}), (z_j^{[0]}, \dots, z_j^{[2^D-1]})$ and $(\text{com}_j^{[0]}, \dots, \text{com}_j^{[2^D-1]})$
 - Compute $X_j \leftarrow \bigoplus_{i=0}^{2^D-1} \text{com}_j^{[i]}$
 - Fold Tree T_j :
 - $(\vec{x}_j^{[0]_i}, \vec{x}_j^{[1]_i})_{i \in [D]} \leftarrow \text{Folding}(\vec{x}_j^{[0]}, \dots, \vec{x}_j^{[2^D-1]})$
 - $(\vec{y}_j^{[0]_i}, \vec{y}_j^{[1]_i})_{i \in [D]} \leftarrow \text{Folding}(\vec{y}_j^{[0]}, \dots, \vec{y}_j^{[2^D-1]})$
 - $(z_j^{[0]_i}, z_j^{[1]_i})_{i \in [D]} \leftarrow \text{Folding}(z_j^{[0]}, \dots, z_j^{[2^D-1]})$
 - $(\text{com}_j^{[0]_i}, \text{com}_j^{[1]_i})_{i \in [D]} \leftarrow \text{Folding}(\text{com}_j^{[0]}, \dots, \text{com}_j^{[2^D-1]})$
 - Let $z \leftarrow z_0^{[0]_0} \oplus z_0^{[1]_0}$
 - Compute offsets $\delta_{\vec{y}_j} \leftarrow \vec{y} \oplus \vec{y}_j^{[0]_0} \oplus \vec{y}_j^{[1]_0}$ and $\delta_{z_j} \leftarrow z \oplus z_j^{[0]_0} \oplus z_j^{[1]_0}$
 5. Compute $h_{\text{com}} \leftarrow \mathcal{H}_3\left(\text{salt}, (\text{com}_j^{[0]_i}, \text{com}_j^{[1]_i})_{(i,j) \in [D] \times [\tau]}, (X_j)_{j \in [\tau]}\right)$
 6. $h_0 \leftarrow \mathcal{H}_0(\text{msg}, h_{\text{com}}, \text{salt}, (\delta_{\vec{y}_j}, \delta_{z_j})_{j \in [\tau]}, (X_j)_{j \in [\tau]})$
 7. $(\vec{\mu}, (\alpha_m)_{m \in [\tau \cdot D]}) \leftarrow \text{PRG}_0(h_0)$
 8. Compute $\vec{A}_{F_x} \leftarrow \sum_{j=0}^{\tau-1} \sum_{i=j \cdot D}^{(j+1) \cdot D-1} \vec{x}_j^{[0]_i} \alpha_i$
 9. Compute $\vec{A}_{F_y} \leftarrow \sum_{j=0}^{\tau-1} \sum_{i=j \cdot D}^{(j+1) \cdot D-1} \vec{y}_j^{[0]_i} \alpha_i$
 10. Compute the coefficients A and B of Φ as in the Response 1 algorithm
 11. Compute $b \leftarrow z + \vec{y} \cdot \vec{\mu}$
 12. For $j \in [\tau]$:
 - Compute $\vec{A}_{F_y}^{(j)} \leftarrow \sum_{i=j \cdot D}^{(j+1) \cdot D-1} \vec{y}_j^{[0]_i} \alpha_i$ and $\vec{A}_{F_z}^{(j)} \leftarrow \sum_{i=j \cdot D}^{(j+1) \cdot D-1} z_j^{[0]_i} \alpha_i$
 - Compute $a^{(j)} \leftarrow \vec{A}_{F_z}^{(j)} + \vec{A}_{F_y}^{(j)} \cdot \vec{\mu}$
 13. $h_1 \leftarrow \mathcal{H}_1(\text{msg}, h_{\text{com}}, \text{salt}, (\delta_{\vec{y}_j}, \delta_{z_j})_{j \in [\tau]}, h_0, A, B, (a^{(j)})_{j \in [\tau]}, b)$
 14. $(i_j^*)_{j \in [\tau]} \leftarrow \text{PRG}_1(h_1)$
 15. Compute the pre-challenge $i_p^* = \sum_{j=0}^{\tau-1} \lfloor i_j^* / 2^{D-1} \rfloor \cdot 2^j$
 16. Compute the pre-tree PPRF key $K_{i_p^*}$ and the PPRF key $K_{i_0^*, \dots, i_{\tau-1}^*}$
 17. Output $\sigma \leftarrow (h_1, \text{salt}, (\delta_{\vec{y}_j})_{j \in [\tau]}, (\delta_{z_j})_{j \in [\tau]^*}, B, b, K_{i_p^*}, K_{i_0^*, \dots, i_{\tau-1}^*}, (X_j)_{j \in [\tau]})$

Fig. 9. SBC_{VOLE} signature scheme - Signing algorithm

respective signature sizes are displayed in Table 1. We note that, unlike the standard SBC scheme [21], using $(D, \tau) = (16, 8)$ and $(D, \tau) = (8, 16)$ does not yield a signature with 128 bits security level, since (6) yields a security parameter

Inputs: Public key $\mathbf{pk} = (\vec{u}, \vec{v})$, a message $\mathbf{msg} \in \{0, 1\}^*$ and a signature $\sigma = (h_1, \mathbf{salt}, (\delta_{y_j})_{j \in [\tau]}, (\delta_{z_j})_{j \in [\tau]}, B, b, K_{i_p}^*, K_{i_0}^*, \dots, K_{i_{\tau-1}}^*, (X_j)_{j \in [\tau]})$

1. $(i_j^*)_{j \in [\tau]} \leftarrow \text{PRG}_1(h_1)$
2. Recompute, expand and fold all trees (with one missing leaf each) using $K_{i_p}^*, K_{i_0}^*, \dots, K_{i_{\tau-1}}^*$ and $\mathbf{salt}$
3. For $(i, j) \in [D] \times [\tau]$, let $b_{i,j}$ denote the known position of the respective hypercube binary sharings
4. For $(i, j) \in [D] \times [\tau]$: Compute $\text{com}_j^{\llbracket 1-b_{i,j} \rrbracket i} \leftarrow X_j \oplus \text{com}_j^{\llbracket b_{i,j} \rrbracket i}$
5. Compute $h'_{\text{com}} \leftarrow \mathcal{H}_3 \left(\mathbf{salt}, \left(\text{com}_j^{\llbracket 0 \rrbracket i}, \text{com}_j^{\llbracket 1 \rrbracket i} \right)_{(i,j) \in [D] \times [\tau]}, (X_j)_{j \in [\tau]} \right)$
6. $\delta_{z_0} \leftarrow 0$
7. $h'_0 \leftarrow \mathcal{H}_0 \left(\mathbf{msg}, h'_{\text{com}}, \mathbf{salt}, (\delta_{y_j}, \delta_{z_j})_{j \in [\tau]}, (X_j)_{j \in [\tau]} \right)$
8. $(\vec{\mu}, (\alpha_m)_{m \in [\tau \cdot D]}) \leftarrow \text{PRG}_0(h'_0)$
9. For $j \in [\tau]$:
 - Compute $\Delta_j^* \leftarrow \sum_{i=j \cdot D}^{(j+1) \cdot D-1} b_{i,j} \alpha_i$
 - Compute $\vec{F}_y^{(j)}(\Delta_j^*) \leftarrow \sum_{i=j \cdot D}^{(j+1) \cdot D-1} \alpha_i \left(\vec{y}_j^{\llbracket b_{i,j} \rrbracket i} + b_{i,j} \delta_{y_j} \right)$
 - Compute $Z^{(j)}(\Delta_j^*) \leftarrow \sum_{i=j \cdot D}^{(j+1) \cdot D-1} \alpha_i \left(z_j^{\llbracket b_{i,j} \rrbracket i} + b_{i,j} \delta_{z_j} \right)$
 - $a^{(j)'} \leftarrow Z^{(j)}(\Delta_j^*) \oplus \vec{F}_y^{(j)}(\Delta_j^*) \cdot \vec{\mu} \oplus b \Delta_j^*$
10. $\Delta^* \leftarrow \sum_{j=0}^{\tau-1} \Delta_j^*$
11. $\vec{F}_x(\Delta^*) \leftarrow \sum_{m=0}^{\tau \cdot D-1} \vec{x}_m^{\llbracket b_m \rrbracket} \alpha_m$
12. $\vec{F}_y(\Delta^*) \leftarrow \sum_{j=0}^{\tau-1} \vec{F}_y^{(j)}(\Delta_j^*)$
13. Compute $\Phi(\Delta^*)$ as

$$\left(\vec{u} \cdot \begin{pmatrix} \vec{F}_x(\Delta^*) \\ 1 \\ 0 \end{pmatrix} \right) \left(\vec{v} \cdot \begin{pmatrix} \vec{F}_y(\Delta^*) \\ 0 \\ 1 \end{pmatrix} \right) - \left(\vec{u} \cdot \begin{pmatrix} \vec{F}_y(\Delta^*) \\ 0 \\ 1 \end{pmatrix} \right) \left(\vec{v} \cdot \begin{pmatrix} \vec{F}_x(\Delta^*) \\ 1 \\ 0 \end{pmatrix} \right)$$
14. $A' \leftarrow \Phi(\Delta^*) - B \Delta^*$
15. $h'_1 \leftarrow \mathcal{H}_1 \left(\mathbf{msg}, h'_{\text{com}}, \mathbf{salt}, (\delta_{y_j}, \delta_{z_j})_{j \in [\tau]}, h'_0, A', B, (a^{(j)'})_{j \in [\tau]}, b \right)$
16. If $h'_1 = h_1$ output **ACCEPT**, otherwise output **REJECT**

Fig. 10. SBC_{VOLE} signature scheme - Verification algorithm

smaller than 127 bits for these cases. This one bit gap comes from the fact that the evaluation point of the quadratic polynomial is selected from a smaller set of values. For this reason, the potential existence of two roots boosts the probability of a cheating prover by a factor of two.

Public key size. The public key of the SBC scheme consists of a seed $\text{seed}_{\mathbf{pk}}$ of size λ and a finite field element in $\mathbb{F}_{q^k}$, using the key generation described in Section 2.2. From the seed, we can derive the values of $\vec{u}$ and $v_0, \dots, v_{n-2}$ using a PRG, while the element $v_{n-1} \in \mathbb{F}_{q^k}$ is sent separately. Therefore, the public key size is 3λ bits. For $\lambda = 128$, this yields a public key size of 48 bytes.

Table 1. SBC_{VOLE} signature size for $N = 2^D$ parties and τ rounds with parameters $q = 2, k = 257$ and $n = 130$ for the security parameter $\lambda = 128$ bits.

D	τ	$\lambda^2 + 5\tau\lambda + 7\lambda$	$ \text{sgn} $
9	15	3 360 B	3 458 B
10	13	3 200 B	3 218 B
11	12	3 120 B	3 170 B
12	11	3 040 B	3 090 B
13	10	2 960 B	2 978 B
15	9	2 880 B	2 978 B

5.3 Proof of Existential Unforgeability

In this section, we provide a proof for existential unforgeability against chosen message attacks (EUF-CMA) of the SBC signature scheme. The adversary, given a public key $\text{pk} = (\vec{u}, \vec{v})$, is allowed to query a signing oracle to sign messages of its choice. The goal of the adversary is to generate a valid message-signature pair (msg, σ) , with the restriction that msg was not queried before. In the security analysis, we also use the notion of existential unforgeability against key-only attacks (EUF-KO), where the adversary only has access to the public key. The formal definitions of these standard cryptographic notions are given below.

Definition 3 (EUF-CMA security). *We say that a signature scheme of the form $(\text{KeyGen}, \text{Sign}, \text{Verify})$ is EUF-CMA-secure, given the security parameter λ , if any PPT algorithm $\mathcal{A}$ has negligible advantage in the EUF-CMA security game, which is defined as*

$$\text{Adv}_{\mathcal{A}}^{\text{EUF-CMA}} = \Pr \left[\begin{array}{l} \text{Verify}(\text{pk}, \text{msg}, \sigma) = 1 \\ \wedge \text{msg} \notin \mathcal{M}_q \end{array} \middle| \begin{array}{l} (\text{sk}, \text{pk}) \leftarrow \text{KeyGen}(\{0, 1\}^\lambda) \\ (\text{msg}, \sigma) \leftarrow \mathcal{A}^{\mathcal{O}(\text{sk}, \text{pk})}(\text{pk}) \end{array} \right],$$

where the notation $\mathcal{A}^{\mathcal{O}(\text{sk}, \text{pk})}$ means that $\mathcal{A}$ has access to a signing oracle $\mathcal{O}(\text{sk}, \text{pk})$ which has access to the private key sk , and $\mathcal{M}_q$ denotes the set of messages that were queried by $\mathcal{A}$.

Definition 4 (EUF-KO security). *We say that a signature scheme of the form $(\text{KeyGen}, \text{Sign}, \text{Verify})$ is EUF-KO-secure, given the security parameter λ , if any PPT algorithm $\mathcal{A}$ has negligible advantage in the EUF-KO security game, which is defined as*

$$\text{Adv}_{\mathcal{A}}^{\text{EUF-KO}} = \Pr \left[\text{Verify}(\text{pk}, \text{msg}, \sigma) = 1 \middle| \begin{array}{l} (\text{sk}, \text{pk}) \leftarrow \text{KeyGen}(\{0, 1\}^\lambda) \\ (\text{msg}, \sigma) \leftarrow \mathcal{A}(\text{pk}) \end{array} \right].$$

The security analysis for our scheme is based on the following assumptions:

- **Hardness of SBC.** We assume that solving the given SBC instance is hard. In particular, any adversary $\mathcal{A}$, given as input a random SBC instance $(\vec{u}, \vec{v})$ and running in time t , outputs a valid solution $(\vec{x}, \vec{y}) \in \text{SBC}[\vec{u}, \vec{v}]$ with probability at most ε_{SBC} . Here, ε_{SBC} and t implicitly depend on the security parameter λ .

- **The Random Oracle Model.** The hash functions used in the protocol are assumed to be random oracles.
- **Secure pseudorandomness.** We assume that the PRGs used in the protocol are $(t, \varepsilon_{\text{PRG}})$ -secure.

For a proof of the following Theorem, see Appendix A.

Theorem 2. *Consider the NSBC signature scheme described in Section 5.1. Assume that the PRGs used are $(t, \varepsilon_{\text{PRG}})$ -secure and that any adversary running in time t has an advantage of at most ε_{SBC} against the SBC problem. Assume that $\mathcal{H}_0, \mathcal{H}_1, \mathcal{H}_2, \mathcal{H}_3$ are random oracles. Let $\mathcal{A}$ be an adversary against the EUF-CMA security of the signature scheme, which has a running time t and makes q_i queries to $\mathcal{H}_i$ for $i \in \{0, 1, 2, 3\}$ and q_s queries to the signing oracle. Then, the advantage of $\mathcal{A}$ in the EUF-CMA game is upper bounded by*

$$\text{Adv}_{\mathcal{A}}^{\text{EUF-CMA}} \leq \frac{4Q^2}{2^{2\lambda}} + \varepsilon_{\text{PRG}} + \varepsilon_{\text{SBC}} + \varepsilon(t),$$

where $Q = q_0 + q_1 + q_2 + q_3 + N_{\mathcal{H}} \cdot q_s$ is the total number of random oracle queries. Here, $N_{\mathcal{H}} = 3\tau \cdot 2^D + 2^{\tau+1} - 2\tau$ is the number of random oracle queries per signature computation and $\varepsilon := \tau^2 2^{-k} + 2^{1-\tau \cdot D}$ is the soundness error of the SBC protocol from Theorem 1.

6 Implementation and Performance

In this section, we provide the signature sizes and running times of the scheme based on our C implementation, which is an adaptation of the artifact from [21]. The total communication cost of the standard SBC paper is $\lambda^2 + 16\tau\lambda + 3\lambda$ bits, while the communication cost of our scheme is $\lambda^2 + 5\tau\lambda + 7\lambda$ bits. As mentioned in Section 2.4, we use AES as the internal cGGM forest derivation function for $\lambda = 128$ in both implementations to improve the running time, which is a technique introduced in [13].

We compare our scheme with the MPCitH signature scheme for SBC proposed in [21] in Table 2. We see that the case $(D, \tau) = (13, 10)$ yields the smallest signature size of 2978 bytes, which results in a decrease of the signature size of $\approx 21\%$ compared to the smallest signature size obtained by the standard SBC signature scheme from [21], i.e., for $(D, \tau) = (16, 8)$. The running time of the smallest SBC_{VOL} instance $(D, \tau) = (13, 10)$ is reduced by 90% compared to the smallest instance of the standard SBC version for $(D, \tau) = (16, 8)$. We also note that the SBC_{VOL} scheme for $(D, \tau) = (9, 15)$ and $(D, \tau) = (15, 9)$ is redundant since it is slower and larger compared to the $(13, 10)$ version.

Remark 5. Improving signature size with variable-sized trees. The main restriction on finding good parameter choices is to find multiples of the desired number of trees, i.e. the desired value of τ , slightly above 128. As seen in Table 2, this leads to very limited good choices. We can improve the accessible range of parameters by allowing the second level trees in the correlated forest to have variable

Table 2. Signature sizes and running times for $\lambda = 128$ using $N = 2^D$ parties and τ rounds with the parameters $q = 2$, $k = 257$ and $n = 130$ for standard SBC and SBC_{VOLE} using an AMD EPYC 9374F processor running at 3.85 GHz. The CPU clock cycles are given in millions of cycles (Mc).

D	τ	standard SBC [21]						SBC_{VOLE}					
		sgn	Sign		Verify		sgn	Sign		Verify		time	cycles
			time	cycles	time	cycles		time	cycles	time	cycles		
8	16	5 436 B	0.77 ms	3.42	0.68 ms	3.00	—	—	—	—	—	—	—
9	15	5 340 B	0.90 ms	3.97	0.81 ms	3.55	3 360 B	2.41 ms	9.29	2.40 ms	9.26	—	—
10	13	4 842 B	1.29 ms	4.54	1.21 ms	4.14	3 200 B	1.58 ms	6.08	1.57 ms	6.05	—	—
11	12	4 665 B	1.60 ms	6.00	1.51 ms	5.59	3 170 B	1.60 ms	6.17	1.59 ms	6.12	—	—
12	11	4 457 B	2.25 ms	8.61	2.15 ms	8.17	3 090 B	1.79 ms	6.88	1.77 ms	6.82	—	—
13	10	4 216 B	3.47 ms	13.5	3.34 ms	12.9	2 978 B	2.21 ms	8.53	2.19 ms	8.43	—	—
15	9	4 087 B	11.3 ms	43.9	10.9 ms	42.6	2 978 B	5.13 ms	19.8	5.0 ms	19.3	—	—
16	8	3 766 B	19.7 ms	77.6	19.2 ms	75.1	—	—	—	—	—	—	—

sizes. For example, with $\tau = 9$, we could use four trees of size 2^{15} and five of size 2^{14} . In this situation, the number of binary sharings would be 130, i.e., the same as in the $\tau = 10$ case. This approach can potentially reduce the signature sizes to the estimated value $\lambda^2 + 5\tau\lambda + 7\lambda$ given in Table 1.

6.1 Using MinMax Commitments to Improve the Signature Size

To reduce the communication cost of the signature scheme, we introduce an alternative way to commit to the cGGM forest. In the basic version from Section 5, we expand each leaf into a total of 5λ bits: λ bits for shares of $\vec{y}$, 2λ bits for shares of z and 2λ bits for the commitment values $\text{com}_j^{[i]}$.

In this section, we introduce an implicit commitment based on the hash values $\text{com}_j^{[i]}$, which are pseudorandom values derived using a random oracle. This reduces the communication cost of the signature scheme by $\tau \cdot 2\lambda$ bits, corresponding to the XOR values $(X_j)_{j \in [\tau]}$. However, this method has an abort probability, which leads to a small (amortized) increase of the expected running time of the signing algorithm. Note that this technique is generically applicable to other MPCitH or VOLEitH signature schemes that use GGM trees and not tied to our SBC protocol.

For each subtree T_j , we compute the maximum value among the hash values $\text{com}_j^{[i]}$ with respect to some fixed ordering (e.g. lexicographical ordering). In particular, let

$$M_{i_j^{\max}} \leftarrow \max\{\text{com}_j^{[0]}, \dots, \text{com}_j^{[2^D-1]}\}$$

for j in $[\tau]$. We denote the indices of these values by $i_j^{\max} \in [2^D]$ for $j \in [\tau]$. Since each commitment value is a pseudorandom value of 2λ bits, the maximum value in each subtree T_j is unique for each $j \in [\tau]$ with overwhelming probability. Indeed, if there are two distinct positions in a subtree with the same value, these correspond to a collision of the hash function used to expand the leaves. This

happens with negligible probability. If the maximum value of a given subtree is not unique, we abort and restart the signing process. In this new version, we replace the computation of the commitment hash with

$$h_{\text{com}} \leftarrow \mathcal{H}_3 \left(\text{salt}, (j, M_{i_j^{\max}}, i_j^{\max})_{j \in [\tau]}, (\delta_{\vec{y}_j})_{j \in [\tau]}, (\delta_{z_j})_{j \in [\tau]^*} \right).$$

After computing the challenges based on this hash as before, the signer obtains the indices $(i_j^*)_{j \in [\tau]}$ using PRG_1 . If one of these challenge indices coincides with the index $i_j^{\max}$ of a maximum value $M_{i_j^{\max}}$ in the corresponding subtree T_j , the signer aborts and restarts the signing process. By the union bound, the probability of this event is at most $\tau \cdot \frac{1}{2^D}$. Otherwise, the signer proceeds as before and sends the signature

$$\sigma \leftarrow \left(h_1, \text{salt}, (\delta_{\vec{y}_j})_{j \in [\tau]}, (\delta_{z_j})_{j \in [\tau]^*}, B, b, K_{i_p^*}, K_{i_0^*, \dots, i_{\tau-1}^*} \right)$$

to the verifier. Note that, compared to the standard version, we do not need to send the values $(X_j)_{j \in [\tau]}$ in this version of the signature.

The verifier proceeds as before and also computes the maximum values among the hashes in each subtree:

$$M'_{i_j^{\max}} \leftarrow \max \{ \text{com}_j^{[0]}, \dots, \text{com}_j^{[i_j^* - 1]}, \text{com}_j^{[i_j^* + 1]}, \dots, \text{com}_j^{[2^D - 1]} \}.$$

Since $i_j^* \neq i_j^{\max}$ for $j \in [\tau]$, the maximum values and their respective indices match the ones computed by the signer. Therefore, the verifier can recompute

$$h'_{\text{com}} \leftarrow \mathcal{H}_3 \left(\text{salt}, (j, M'_{i_j^{\max}}, i_j^{\max})_{j \in [\tau]}, (\delta_{\vec{y}_j})_{j \in [\tau]}, (\delta_{z_j})_{j \in [\tau]^*} \right)$$

and proceed with the protocol as before.

This version of the signature uses a string of $2\tau\lambda$ bits to commit to the cGGM forest. In the random oracle model, finding two trees with the same commitment string is virtually impossible. In some sense, $2\tau\lambda$ bits is even overkill for our purpose. Therefore, to improve the signing speed, we can instead use the global maximum across all τ subtrees as an implicit commitment to the cGGM forest. In particular, we can compute

$$M^{\max} \leftarrow \max \bigcup_{j \in [\tau]} \{ \text{com}_j^{[0]}, \dots, \text{com}_j^{[2^D - 1]} \}$$

and adapt the computation of the commitment hash accordingly:

$$h_{\text{com}} \leftarrow \mathcal{H}_3 \left(\text{salt}, M^{\max}, (\delta_{\vec{y}_j})_{j \in [\tau]}, (\delta_{z_j})_{j \in [\tau]^*} \right).$$

This version has an abort probability of $\frac{1}{2^D}$. However, we only use a string of 2λ bits to commit to the cGGM forest if we do that. To find a middle ground between these two versions, we also include the global minimum into the commitment hash computation. In particular, we compute

$$M^{\min} \leftarrow \min \bigcup_{j \in [\tau]} \{ \text{com}_j^{[0]}, \dots, \text{com}_j^{[2^D - 1]} \}$$

and compute the commitment hash as

$$h_{\text{com}} \leftarrow \mathcal{H}_3 \left(\text{salt}, M^{\min}, M^{\max}, (\delta_{\bar{y}_j})_{j \in [\tau]}, (\delta_{z_j})_{j \in [\tau]^*} \right).$$

Similar to before, if one of the challenge indices $(i_j^*)_{j \in [\tau]}$ coincides with the index of the minimum or with the index of the maximum or if the extrema are not unique, we abort. In this version, the verifier recomputes the minimum and maximum values among the commitment hashes as

$$M^{\min'} \leftarrow \min \bigcup_{j \in [\tau]} \left(\bigcup_{i \in [2^D] \setminus \{i_j^*\}} \{\text{com}_j[\llbracket i \rrbracket]\} \right),$$

$$M^{\max'} \leftarrow \max \bigcup_{j \in [\tau]} \left(\bigcup_{i \in [2^D] \setminus \{i_j^*\}} \{\text{com}_j[\llbracket i \rrbracket]\} \right).$$

Since i_j^* does not equal to the position of the minimum or maximum for $j \in [\tau]$, the recomputed minimum and maximum value match the ones computed by the signer and the verifier can proceed as before.

The abort probability of this version is upper bounded by $\frac{2}{2^D}$ and we use a string of size 4λ bits to commit to the cGGM forest. The signature size of this new version is $\lambda^2 + 3\tau\lambda + 7\lambda$ bits, compared to $\lambda^2 + 5\tau\lambda + 7\lambda$ bits for the basic version. We denote the signature scheme with this MinMax commitment using the superscript $\text{SBC}_{\text{VOLE}}^M$. We provide a comparison of signature sizes and running times of this new version compared to the basic version in Table 3. The pseudocode of the $\text{SBC}_{\text{VOLE}}^M$ version is deferred to Appendix B.

Table 3. Signature sizes and running times for $\lambda = 128$ using $N = 2^D$ parties and τ rounds with the parameters $q = 2$, $k = 257$ and $n = 130$ for SBC_{VOLE} and $\text{SBC}_{\text{VOLE}}^M$ using an AMD EPYC 9374F processor running at 3.85 GHz.

Parameters		SBC_{VOLE}			$\text{SBC}_{\text{VOLE}}^M$			
D	τ	sgn	Sign	Verify	sgn	Pr[abort]	Sign (average)	Verify
9	15	3 360 B	2.41 ms	2.40 ms	2 978 B	0.39 %	2.49 ms	2.40 ms
10	13	3 200 B	1.58 ms	1.57 ms	2 802 B	0.20 %	1.61 ms	1.57 ms
11	12	3 170 B	1.60 ms	1.59 ms	2 786 B	0.10 %	1.61 ms	1.59 ms
12	11	3 090 B	1.79 ms	1.77 ms	2 738 B	0.05 %	1.80 ms	1.77 ms
13	10	2 978 B	2.21 ms	2.19 ms	2 658 B	0.02 %	2.22 ms	2.19 ms
15	9	2 978 B	5.13 ms	5.02 ms	2 690 B	0.01 %	5.14 ms	5.02 ms

6.2 Comparison with other VOLEitH Signature Schemes

In this section, we compare our SBC signature scheme with other post-quantum VOLEitH based signatures from the literature. These include the NIST proposal using the security of AES and the Even-Mansour AES variant (EM-AES) [4]. The ReSolved signature scheme uses the Regular Syndrome Decoding (RSD)

problem over $\mathbb{F}_2$. The signature scheme in [12] is based on the Multivariate Quadratic (MQ) problem, while [8] is based on the Permuted Kernel Problem (PKP). In [10], the authors construct signature schemes based on the RSD_s problem (which is a variant of RSD) and MinRank.

Table 4. Size comparison of the SBC_{VOLE} signature with other schemes from the literature for the security level of $\lambda = 128$ bits. For each scheme, we use the smallest signature size for this security level that is specified in the respective documentation.

Scheme	Assumption	pk	sgn
FAEST [4]	AES	32 B	5 006 B
FAEST-EM [4]	EM-AES	32 B	4 566 B
ReSolveD [14]	$\text{RSD over } \mathbb{F}_2$	96 B	3 916 B
VOLEitH from MQ [12]	MQ	38 B	3 792 B
VOLEitH from PKP [8]	PKP	102 B	3 686 B
Dual Support Decomposition in the Head [10]	RSD_s	75 B	2 851 B
Dual Support Decomposition in the Head [10]	MinRank	73 B	2 813 B
standard MPCitH SBC [21]	SBC	48 B	3 766 B
SBC_{VOLE}	SBC	48 B	2 978 B
$\text{SBC}_{\text{VOLE}}^M$	SBC	48 B	2 658 B

References

1. Aguilar-Melchor, C., Gama, N., Howe, J., Hülsing, A., Joseph, D., Yue, D.: The return of the SDitH. In: Hazay, C., Stam, M. (eds.) EUROCRYPT 2023, Part V. LNCS, vol. 14008, pp. 564–596. Springer, Cham (Apr 2023). https://doi.org/10.1007/978-3-031-30589-4_20
2. American National Standards Institute, Inc.: ANSI X9.62 public key cryptography for the financial services industry: the elliptic curve digital signature algorithm (ECDSA) (Nov 16, 2005), <https://standards.globalspec.com/std/1955141/ANSI%20X9.62>
3. Aragon, N., Bardet, M., Bidoux, L., Chi-Domínguez, J., Dyseryn, V., Feneuil, T., Gaborit, P., Neveu, R., Rivain, M., Tillich, J.: MIRA. Tech. rep., National Institute of Standards and Technology (2023), available at <https://csrc.nist.gov/Projects/pqc-dig-sig/round-1-additional-signatures>
4. Baum, C., Braun, L., Delpech de Saint Guilhem, C., Kloof, M., Majenz, C., Mukherjee, S., Orsini, E., Ramacher, S., Rechberger, C., Roy, L., Scholl, P.: FAEST. Tech. rep., National Institute of Standards and Technology (2024), available at <https://csrc.nist.gov/Projects/pqc-dig-sig/round-2-additional-signatures>
5. Baum, C., Braun, L., Delpech de Saint Guilhem, C., Kloof, M., Orsini, E., Roy, L., Scholl, P.: Publicly verifiable zero-knowledge and post-quantum signatures from VOLE-in-the-head. In: Handschuh, H., Lysyanskaya, A. (eds.) CRYPTO 2023, Part V. LNCS, vol. 14085, pp. 581–615. Springer, Cham (Aug 2023). https://doi.org/10.1007/978-3-031-38554-4_19
6. Baum, C., Delpech de Saint Guilhem, C., Kales, D., Orsini, E., Scholl, P., Zaverucha, G.: Banquet: Short and fast signatures from AES. In: Garay, J. (ed.) PKC 2021, Part I. LNCS, vol. 12710, pp. 266–297. Springer, Cham (May 2021). https://doi.org/10.1007/978-3-030-75245-3_11

7. Benadjila, R., Feneuil, T., Rivain, M.: MQ on my mind: Post-quantum signatures from the non-structured multivariate quadratic problem. In: 2024 IEEE European Symposium on Security and Privacy. pp. 468–485. IEEE Computer Society Press (Jul 2024). <https://doi.org/10.1109/EuroSP60621.2024.00032>
8. Bettaieb, S., Bidoux, L., Gaborit, P., Kulkarni, M.: Modelings for generic PoK and applications: Shorter SD and PKP based signatures. Cryptology ePrint Archive, Paper 2024/1668 (2024), <https://eprint.iacr.org/2024/1668>
9. Bettale, L., Kahrobaei, D., Perret, L., Verbel, J.: Biscuit. Tech. rep., National Institute of Standards and Technology (2023), available at <https://csrc.nist.gov/Projects/pqc-dig-sig/round-1-additional-signatures>
10. Bidoux, L., Feneuil, T., Gaborit, P., Neveu, R., Rivain, M.: Dual support decomposition in the head: Shorter signatures from rank SD and MinRank. In: Chung, K.M., Sasaki, Y. (eds.) ASIACRYPT 2024, Part II. LNCS, vol. 15485, pp. 38–69. Springer, Singapore (Dec 2024). https://doi.org/10.1007/978-981-96-0888-1_2
11. Boneh, D., Waters, B.: Constrained pseudorandom functions and their applications. In: Sako, K., Sarkar, P. (eds.) ASIACRYPT 2013, Part II. LNCS, vol. 8270, pp. 280–300. Springer, Berlin, Heidelberg (Dec 2013). https://doi.org/10.1007/978-3-642-42045-0_15
12. Bui, D.: Shorter VOLEith signature from multivariate quadratic. Cryptology ePrint Archive, Report 2024/465 (2024), <https://eprint.iacr.org/2024/465>
13. Bui, D., Carozza, E., Couteau, G., Goudarzi, D., Joux, A.: Faster signatures from MPC-in-the-head. In: Chung, K.M., Sasaki, Y. (eds.) ASIACRYPT 2024, Part I. LNCS, vol. 15484, pp. 396–428. Springer, Singapore (Dec 2024). https://doi.org/10.1007/978-981-96-0875-1_13
14. Cui, H., Liu, H., Yan, D., Yang, K., Yu, Y., Zhang, K.: ReSolveD: Shorter signatures from regular syndrome decoding and VOLE-in-the-head. In: Tang, Q., Teague, V. (eds.) PKC 2024, Part I. LNCS, vol. 14601, pp. 229–258. Springer, Cham (Apr 2024). https://doi.org/10.1007/978-3-031-57718-5_8
15. Feneuil, T.: Building MPCitH-based signatures from MQ, MinRank, rank SD and PKP. Cryptology ePrint Archive, Report 2022/1512 (2022), <https://eprint.iacr.org/2022/1512>
16. Feneuil, T., Joux, A., Rivain, M.: Syndrome decoding in the head: Shorter signatures from zero-knowledge proofs. In: Dodis, Y., Shrimpton, T. (eds.) CRYPTO 2022, Part II. LNCS, vol. 13508, pp. 541–572. Springer, Cham (Aug 2022). https://doi.org/10.1007/978-3-031-15979-4_19
17. Feneuil, T., Rivain, M.: MQOM — MQ on my Mind. Tech. rep., National Institute of Standards and Technology (2023), available at <https://csrc.nist.gov/Projects/pqc-dig-sig/round-1-additional-signatures>
18. Fiat, A., Shamir, A.: How to prove yourself: Practical solutions to identification and signature problems. In: Odlyzko, A.M. (ed.) CRYPTO’86. LNCS, vol. 263, pp. 186–194. Springer, Berlin, Heidelberg (Aug 1987). https://doi.org/10.1007/3-540-47721-7_12
19. Goldreich, O., Goldwasser, S., Micali, S.: How to construct random functions. Journal of the ACM **33**(4), 792–807 (Oct 1986). <https://doi.org/10.1145/6490.6503>
20. Guo, X., Yang, K., Wang, X., Zhang, W., Xie, X., Zhang, J., Liu, Z.: Half-tree: Halving the cost of tree expansion in COT and DPF. In: Hazay, C., Stam, M. (eds.) EUROCRYPT 2023, Part I. LNCS, vol. 14004, pp. 330–362. Springer, Cham (Apr 2023). https://doi.org/10.1007/978-3-031-30545-0_12
21. Huth, J., Joux, A.: MPC in the head using the subfield bilinear collision problem. In: Reyzin, L., Stebila, D. (eds.) CRYPTO 2024, Part I. LNCS, vol. 14920, pp. 39–70. Springer, Cham (Aug 2024). https://doi.org/10.1007/978-3-031-68376-3_2

22. Ishai, Y., Kushilevitz, E., Ostrovsky, R., Sahai, A.: Zero-knowledge from secure multiparty computation. In: Johnson, D.S., Feige, U. (eds.) 39th ACM STOC. pp. 21–30. ACM Press (Jun 2007). <https://doi.org/10.1145/1250790.1250794>
23. Joux, A., Pierrot, C.: Algorithmic aspects of elliptic bases in finite field discrete logarithm algorithms (2024). <https://doi.org/10.3934/amc.2022085>, <https://www.aims sciences.org/article/id/6368515a6aa93c395b347970>
24. Kales, D., Zaverucha, G.: An attack on some signature schemes constructed from five-pass identification schemes. In: Krenn, S., Shulman, H., Vaudenay, S. (eds.) CANS 20. LNCS, vol. 12579, pp. 3–22. Springer, Cham (Dec 2020). https://doi.org/10.1007/978-3-030-65411-5_1
25. Katz, J., Kolesnikov, V., Wang, X.: Improved non-interactive zero knowledge with applications to post-quantum signatures. Cryptology ePrint Archive, Report 2018/475 (2018), <https://eprint.iacr.org/2018/475>
26. Kim, S., Lee, B., Son, M.: Shorter VOLE-in-the-head-based signatures from vector semi-commitment. Cryptology ePrint Archive, Paper 2025/1077 (2025), <https://eprint.iacr.org/2025/1077>
27. Rivest, R.L., Shamir, A., Adleman, L.M.: A method for obtaining digital signatures and public-key cryptosystems. Communications of the Association for Computing Machinery **21**(2), 120–126 (Feb 1978). <https://doi.org/10.1145/359340.359342>
28. Roy, L.: SoftSpokenOT: Quieter OT extension from small-field silent VOLE in the minicrypt model. In: Dodis, Y., Shrimpton, T. (eds.) CRYPTO 2022, Part I. LNCS, vol. 13507, pp. 657–687. Springer, Cham (Aug 2022). https://doi.org/10.1007/978-3-031-15802-5_23
29. Schnorr, C.P.: Efficient identification and signatures for smart cards. In: Brassard, G. (ed.) CRYPTO’89. LNCS, vol. 435, pp. 239–252. Springer, New York (Aug 1990). https://doi.org/10.1007/0-387-34805-0_22
30. Yang, K., Sarkar, P., Weng, C., Wang, X.: QuickSilver: Efficient and affordable zero-knowledge proofs for circuits and polynomials over any field. In: Vigna, G., Shi, E. (eds.) ACM CCS 2021. pp. 2986–3001. ACM Press (Nov 2021). <https://doi.org/10.1145/3460120.3484556>

A Proof of Existential Unforgeability Against Chosen Message Attacks

We provide a proof of Theorem 2 below. This proof is an adaptation of the EUF-CMA proof of the Banquet signature scheme [6] and follows the standard hybrid argument proof technique used in MPCitH signature schemes like [7,16,12].

Proof. First, we consider the EUF-KO game and upper-bound the advantage of the adversary in this game. For simplicity, we assume that the public key and the output from the PRGs are truly random values, instead of being pseudorandomly generated. Later in the proof, we account for the pseudorandomness in the real signature scheme by relaxing this assumption.

EUF-KO security. ($\text{Adv}_{\mathcal{A}}^{\text{EUF-KO}}$) Assume an adversary $\mathcal{A}$ with running time t wins the EUF-KO security game with probability $\varepsilon_{\text{EUF-KO}}$. We use $\mathcal{A}$ to construct a reduction $\mathcal{R}$ that solves the SBC problem in time t with probability ε_{SBC} . This reduction $\mathcal{R}$, given the public key $(\vec{u}, \vec{v})$, interacts with $\mathcal{A}$ by simulating the EUF-KO game. For any hash query by $\mathcal{A}$, the reduction samples a random value and returns it as a response. For the whole interaction, $\mathcal{R}$ keeps a list $\mathcal{Q}$ of all random oracle queries q and a list L with the corresponding answers h . Given a fresh query $q \notin \mathcal{Q}$, $\mathcal{R}$ samples a random value h . If $h \in L$, $\mathcal{R}$ aborts, otherwise, h is added to L . Moreover, for each query to $\mathcal{H}_1$ of the form

$$h_1 \leftarrow \mathcal{H}_1 \left(\text{msg}, h_{\text{com}}, \text{salt}, (\delta_{\vec{y}_j}, \delta_{z_j})_{j \in [\tau]}, h_0, A, B, \left(a^{(j)} \right)_{j \in [\tau]}, b \right),$$

$\mathcal{R}$ also adds h_0 and h_{com} to L . It follows that

$$\Pr[\mathcal{R} \text{ aborts}] \leq q_{\mathcal{H}} \cdot \frac{\max |L|}{2^{2\lambda}} = q_{\mathcal{H}} \cdot \frac{(q_0 + 3q_1 + q_2 + q_3)}{2^{2\lambda}} \leq \frac{3q_{\mathcal{H}}^2}{2^{2\lambda}}, \quad (7)$$

where $q_{\mathcal{H}} = q_0 + q_1 + q_2 + q_3$.

During the interaction, $\mathcal{R}$ also keeps track of a list $\mathcal{L}_{\text{cand}}$ of the candidate witnesses that can be derived from the queries of $\mathcal{A}$ to $\mathcal{H}_2$ and $\mathcal{H}_3$.

In particular, from the internal nodes of the cGGM tree derivation of the form $\mathcal{H}_2(\text{salt} \parallel j \parallel \text{pos} \parallel \text{node value})$, we can derive the leaf values $\vec{x}_j^{[i]}$. From these shares, $\mathcal{R}$ can also derive $\vec{y}_j^{[i]}$ and add $\vec{x} = \sum_{i=0}^{N-1} \vec{x}_j^{[i]}$ to $\mathcal{L}_{\text{cand}}$. After receiving a valid (msg, σ) -pair from $\mathcal{A}$, the reduction checks if there is a candidate witness in $\mathcal{L}_{\text{cand}}$ such that $(\vec{x}, \vec{y}) \in \text{SBC}[\vec{u}, \vec{v}]$, where $\vec{y} = \sum_{i=0}^{N-1} \vec{y}_j^{[i]} + \delta_{\vec{y}_j}$. If this is the case, then $\mathcal{R}$ returns $(\vec{x}, \vec{y})$, otherwise $\mathcal{R}$ returns $\perp$.

By the union bound, we have that

$$\begin{aligned} \Pr[\mathcal{A} \text{ wins}] &= \Pr[\mathcal{A} \text{ wins} \wedge \mathcal{R} \text{ aborts}] + \Pr[\mathcal{A} \text{ wins} \wedge \mathcal{R} \text{ returns } \perp] \\ &\quad + \Pr[\mathcal{A} \text{ wins} \wedge \mathcal{R} \text{ returns } (\vec{x}, \vec{y})] \\ &\leq \Pr[\mathcal{R} \text{ aborts}] + \Pr[\mathcal{A} \text{ wins} \mid \mathcal{R} \text{ returns } \perp] + \Pr[\mathcal{R} \text{ returns } (\vec{x}, \vec{y})]. \end{aligned}$$

By (7) and the assumed hardness of SBC, this yields

$$\text{Adv}_{\mathcal{A}}^{\text{EUF-KO}} \leq \frac{3q_{\mathcal{H}}^2}{2^{2\lambda}} + \Pr[\mathcal{A} \text{ wins} \mid \mathcal{R} \text{ returns } \perp] + \varepsilon_{\text{SBC}}.$$

Next, we analyze the conditional probability $\Pr[\mathcal{A} \text{ wins} \mid \mathcal{R} \text{ returns } \perp]$. If $\mathcal{R}$ returns $\perp$, this means that there is no solution to the SBC problem that is obtained from the internal queries to $\mathcal{H}_2$ in the cGGM construction by the adversary. If no such queries exists, the adversary can only output a valid signature, if all consistency checks verify and the evaluation point Δ^* is a root of the degree 2 polynomial $\Phi(\Delta) - B\Delta - A$. As seen in the proof of Theorem 1, this probability is at most $\varepsilon = \tau^2 2^{-k} + 2^{1-\tau \cdot D}$. In total, we can therefore bound the advantage of $\mathcal{A}$ in the existential unforgeability game against key-only attacks by

$$\text{Adv}_{\mathcal{A}}^{\text{EU-F-KO}} \leq \frac{3q_{\mathcal{H}}^2}{2^{2\lambda}} + \varepsilon(t) + \varepsilon_{\text{SBC}}.$$

We use a sequence of modified games, based on the reduction $\mathcal{R}$ above, to upper-bound the advantage of the adversary in the EUF-CMA game and use $\varepsilon_{\text{Game}_i}$ to denote the probability that $\mathcal{R}$ outputs a solution $(\vec{x}, \vec{y}) \in \text{SBC}[\vec{u}, \vec{v}]$ in game i .

Game 1. In this game, the reduction also answers signature queries from $\mathcal{A}$ by using the signing oracle $\mathcal{O}(\text{sk}, \text{pk})$. For each signing query, the reduction uses the same method as before to answer the hash queries, while the signature is computed as in the Signing algorithm described in Figure 9. The total number of random oracle queries during the interaction is therefore $Q = q_0 + q_1 + q_2 + q_3 + N_{\mathcal{H}} \cdot q_s$, where the total number of hash queries per signing query is $N_{\mathcal{H}} = 3\tau \cdot 2^D + 2^{\tau+1} - 2\tau$ (one query to $\mathcal{H}_0$ and $\mathcal{H}_1$ each, $2^{\tau+1} - 2$ queries to $\mathcal{H}_2$ in the pre-tree computation, $2^{D+1} - 2$ queries to $\mathcal{H}_2$ for each of the τ second layer trees and $\tau \cdot 2^D$ for the commitment values). Assume that an adversary $\mathcal{A}$ with running time t wins the EUF-CMA security game with probability $\varepsilon_{\text{EUF-CMA}}$. By doing the same union bound argument as in the key-only game, we obtain the upper bound

$$\text{Adv}_{\mathcal{A}}^{\text{EUF-CMA}} \leq \frac{3Q^2}{2^{2\lambda}} + \varepsilon(t) + \varepsilon_{\text{Game}_1}. \quad (8)$$

Game 2. We modify the first game. Upon answering signature queries, the signing oracle $\mathcal{O}(\text{sk}, \text{pk})$ is replaced by a simulator $\mathcal{S}(\text{pk})$, which does not have the secret key sk as input. This simulator behaves similar to the one presented in the original SBC paper [21], with some adaptations for the new VOLEitH based protocol. In particular, during the interaction, $\mathcal{R}$ saves a list L_{salt} of all salt values used in random oracle queries of the form

$$\begin{aligned} h_1 &\leftarrow \mathcal{H}_1 \left(\text{msg}, h_{\text{com}}, \text{salt}, (\delta_{\vec{y}_j}, \delta_{z_j})_{j \in [\tau]}, h_0, A, B, \left(a^{(j)} \right)_{j \in [\tau]}, b \right) \text{ and} \\ h_0 &\leftarrow \mathcal{H}_0 \left(\text{msg}, h_{\text{com}}, \text{salt}, (\delta_{\vec{y}_j}, \delta_{z_j})_{j \in [\tau]}, (X_j)_{j \in [\tau]} \right). \end{aligned}$$

First, the simulator picks random values for salt and $R_{\vec{x}}$ and aborts if the salt was used in a previous cGGM tree construction for a signature. This way, if $\mathcal{S}$ does not abort, we can view the shares of $\vec{x}$ generated by the tree as random values, since they come from a new cGGM forest. Since the simulator $\mathcal{S}$ knows the challenges in advance, it can generate a signature with perfect distribution without knowing sk .

In particular, $\mathcal{S}$ picks random values h_0, h_1 for the random oracle outputs of $\mathcal{H}_0$ and $\mathcal{H}_1$ in advance, instead of computing them during the signing process. In the Signing algorithm from Figure 9, the output of $\mathcal{H}_0$ is set to h_0 and the output of $\mathcal{H}_1$ is set to h_1 . Using the PRGs, these values are expanded to obtain the challenges $(\vec{\mu}, (\alpha_m)_{m \in [\tau \cdot D]})$ and $(i_j^*)_{j \in [\tau]}$. The simulator then proceeds to derive the shares of $\vec{x}$ and $\vec{y}$ as in the original Signing algorithm, with the difference that $\vec{x}$ and the offset values $(\delta_{\vec{y}_j})_{j \in [\tau]}$ are picked at random. Therefore, $\vec{x}$ and the derived $\vec{y} = \sum_{i=0}^{N-1} \vec{y}_j^{[i]} + \delta_{\vec{y}_j}$ solve the SBC problem only with negligible probability. The simulator can then use the Verification algorithm to compute the expected values of A, B, b and $(\delta_{z_j})_{j \in [\tau]^*}$. Using this simulator, the shares revealed in the signature are uniformly distributed under the condition that $\Phi(\Delta^*) = A + B\Delta^*$, i.e., they have the same distribution as the shares obtained from an honest signature execution. If the algorithms do not abort, the obtained signatures from $\mathcal{S}(\text{pk})$ and $\mathcal{O}(\text{sk}, \text{pk})$ are therefore identically distributed. Moreover, the probability that the algorithms abort because a hash collision (in list L) was found is equal for both cases. The simulator might also abort if there is a collision for $\text{salt} \leftarrow \{0, 1\}^{2\lambda}$, which occurs with probability at most $\frac{(q_0 + q_1 + q_2 + q_3 + q_s)}{2^{2\lambda}}$ per signing query. Therefore, by comparing Game 1 and Game 2, we see that

$$|\varepsilon_{\text{Game}_2} - \varepsilon_{\text{Game}_1}| \leq \frac{q_s(q_0 + q_1 + q_2 + q_3 + q_s)}{2^{2\lambda}} \leq \frac{Q^2}{2^{2\lambda}}. \quad (9)$$

From (8) and (9), we obtain

$$\text{Adv}_{\mathcal{A}}^{\text{EUF-CMA}} \leq \varepsilon_{\text{Game}_2} + \frac{4Q^2}{2^{2\lambda}} + \varepsilon(t). \quad (10)$$

Game 3. In this game, we replace the idealized true randomness generator from the previous games by the real PRGs used in the signature scheme. By assumption, each PRG is $(t, \varepsilon_{\text{PRG}})$ -indistinguishable from true randomness. Also, we pseudorandomly generate the public key $(\vec{u}, \vec{v})$ based on a seed seed_{pk} , as described in Section 5.2. Therefore, this game represents the interaction of $\mathcal{R}$ with the real SBC signature scheme from Figure 9. The probability that $\mathcal{R}$ recovers the secret key $(\vec{x}, \vec{y}) \in \text{SBC}[\vec{u}, \vec{v}]$ in Game 3 is therefore $\varepsilon_{\text{Game}_3} = \varepsilon_{\text{SBC}}$. Since we only changed the true randomness from Game 2 into pseudorandomness, we have

$$|\varepsilon_{\text{Game}_3} - \varepsilon_{\text{Game}_2}| \leq \varepsilon_{\text{PRG}}. \quad (11)$$

Finally, by combining (10) and (11), we obtain the claimed upper bounds as

$$\text{Adv}_{\mathcal{A}}^{\text{EUF-CMA}} \leq \frac{4Q^2}{2^{2\lambda}} + \varepsilon_{\text{PRG}} + \varepsilon_{\text{SBC}} + \varepsilon(t).$$

B Pseudocode of the $\text{SBC}_{\text{VOLE}}^M$ Version

Inputs: Secret key $\text{sk} = (\vec{x}, \vec{y})$, public key $\text{pk} = (\vec{u}, \vec{v})$ and a message $\text{msg} \in \{0, 1\}^*$

1. Sample $R_{\vec{x}} \xleftarrow{\$} (\mathbb{F}_2)^{n-2}$ and $\text{salt} \xleftarrow{\$} \{0, 1\}^{2\lambda}$ for the cGGM tree derivation
2. Compute pre-tree: $(\vec{x}^{[0]}, \dots, \vec{x}^{[2^\tau-1]}) \leftarrow \text{cGGM}_{\text{salt}}(R_{\vec{x}}, R_{\vec{x}} \oplus \vec{x}, 2^\tau)$
3. Fold pre-tree: $(\vec{x}^{[0]_j}, \vec{x}^{[1]_j})_{j \in [\tau]} \leftarrow \text{Folding}(\vec{x}^{[0]}, \dots, \vec{x}^{[2^\tau-1]})$
4. For $j \in [\tau]$:
 - Get Tree T_j : $(\vec{x}_j^{[0]}, \dots, \vec{x}_j^{[2^D-1]}) \leftarrow \text{cGGM}_{\text{salt}}(\vec{x}^{[0]_j}, \vec{x}^{[1]_j}, 2^D)$
 - Expand T_j and get $(\vec{y}_j^{[0]}, \dots, \vec{y}_j^{[2^D-1]})$, $(z_j^{[0]}, \dots, z_j^{[2^D-1]})$ and $(\text{com}_j^{[0]}, \dots, \text{com}_j^{[2^D-1]})$
 - Fold Tree T_j :
 - $(\vec{x}_j^{[0]_i}, \vec{x}_j^{[1]_i})_{i \in [D]} \leftarrow \text{Folding}(\vec{x}_j^{[0]}, \dots, \vec{x}_j^{[2^D-1]})$
 - $(\vec{y}_j^{[0]_i}, \vec{y}_j^{[1]_i})_{i \in [D]} \leftarrow \text{Folding}(\vec{y}_j^{[0]}, \dots, \vec{y}_j^{[2^D-1]})$
 - $(z_j^{[0]_i}, z_j^{[1]_i})_{i \in [D]} \leftarrow \text{Folding}(z_j^{[0]}, \dots, z_j^{[2^D-1]})$
 - Let $z \leftarrow z_0^{[0]_0} \oplus z_0^{[1]_0}$
 - Compute offsets $\delta_{\vec{y}_j} \leftarrow \vec{y} \oplus \vec{y}_j^{[0]_0} \oplus \vec{y}_j^{[1]_0}$ and $\delta_{z_j} \leftarrow z \oplus z_j^{[0]_0} \oplus z_j^{[1]_0}$
5. $M^{\min} \leftarrow \min \bigcup_{j \in [\tau]} \{\text{com}_j^{[0]}, \dots, \text{com}_j^{[2^D-1]}\}$
6. $M^{\max} \leftarrow \max \bigcup_{j \in [\tau]} \{\text{com}_j^{[0]}, \dots, \text{com}_j^{[2^D-1]}\}$
7. If $M^{\min}$ or $M^{\max}$ are not unique: **ABORT**
8. $h_{\text{com}} \leftarrow \mathcal{H}_3(\text{salt}, M^{\min}, M^{\max}, (\delta_{\vec{y}_j})_{j \in [\tau]}, (\delta_{z_j})_{j \in [\tau]^*})$
9. $h_0 \leftarrow \mathcal{H}_0(\text{msg}, h_{\text{com}}, \text{salt}, (\delta_{\vec{y}_j}, \delta_{z_j})_{j \in [\tau]})$
10. $(\vec{\mu}, (\alpha_m)_{m \in [\tau \cdot D]}) \leftarrow \text{PRG}_0(h_0)$
11. Compute $\vec{A}_{F_x} \leftarrow \sum_{j=0}^{\tau-1} \sum_{i=j \cdot D}^{(j+1) \cdot D-1} \vec{x}_j^{[0]_i} \alpha_i$
12. Compute $\vec{A}_{F_y} \leftarrow \sum_{j=0}^{\tau-1} \sum_{i=j \cdot D}^{(j+1) \cdot D-1} \vec{y}_j^{[0]_i} \alpha_i$
13. Compute the coefficients A and B of Φ as in the Response 1 algorithm
14. Compute $b \leftarrow z + \vec{y} \cdot \vec{\mu}$
15. For $j \in [\tau]$:
 - Compute $\vec{A}_{F_y}^{(j)} \leftarrow \sum_{i=j \cdot D}^{(j+1) \cdot D-1} \vec{y}_j^{[0]_i} \alpha_i$ and $\vec{A}_{F_z}^{(j)} \leftarrow \sum_{i=j \cdot D}^{(j+1) \cdot D-1} z_j^{[0]_i} \alpha_i$
 - Compute $a^{(j)} \leftarrow \vec{A}_{F_z}^{(j)} + \vec{A}_{F_y}^{(j)} \cdot \vec{\mu}$
16. $h_1 \leftarrow \mathcal{H}_1(\text{msg}, h_{\text{com}}, \text{salt}, (\delta_{\vec{y}_j}, \delta_{z_j})_{j \in [\tau]}, h_0, A, B, (a^{(j)})_{j \in [\tau]}, b)$
17. $(i_j^*)_{j \in [\tau]} \leftarrow \text{PRG}_1(h_1)$
18. If there is an index $j \in [\tau]$ with $i_j^* = i^{\min}$ or $i_j^* = i^{\max}$: **ABORT**
19. Compute the pre-challenge $i_p^* = \sum_{j=0}^{\tau-1} \lfloor i_j^* / 2^{D-1} \rfloor \cdot 2^j$
20. Compute the pre-tree PPRF key $K_{i_p^*}$ and the PPRF key $K_{i_0^*, \dots, i_{\tau-1}^*}$
21. Output $\sigma \leftarrow (h_1, \text{salt}, (\delta_{\vec{y}_j})_{j \in [\tau]}, (\delta_{z_j})_{j \in [\tau]^*}, B, b, K_{i_p^*}, K_{i_0^*, \dots, i_{\tau-1}^*})$

Fig. 11. $\text{SBC}_{\text{VOLE}}^M$ signature scheme - Signing algorithm

We provide the adapted pseudocode of our signature scheme based on the modifications introduced in Section 6.1. The Signing algorithm is given in Figure 11, while the Verification algorithm is given in Figure 12.

Inputs: Public key $\text{pk} = (\vec{u}, \vec{v})$, a message $\text{msg} \in \{0, 1\}^*$ and a signature $\sigma = (h_1, \text{salt}, (\delta_{y_j})_{j \in [\tau]}, (\delta_{z_j})_{j \in [\tau]^*}, B, b, K_{i_p}^*, K_{i_0}^*, \dots, K_{i_{\tau-1}}^*)$

1. $(i_j^*)_{j \in [\tau]} \leftarrow \text{PRG}_1(h_1)$
2. Recompute, expand and fold all trees (with one missing leaf each) using $K_{i_p}^*, K_{i_0}^*, \dots, K_{i_{\tau-1}}^*$ and salt
3. $M^{\min'} \leftarrow \min \bigcup_{j \in [\tau]} \left(\bigcup_{i \in [2^D] \setminus \{i_j^*\}} \{\text{com}_j^{\llbracket i \rrbracket}\} \right)$
4. $M^{\max'} \leftarrow \max \bigcup_{j \in [\tau]} \left(\bigcup_{i \in [2^D] \setminus \{i_j^*\}} \{\text{com}_j^{\llbracket i \rrbracket}\} \right)$
5. For $(i, j) \in [D] \times [\tau]$, let $b_{i,j}$ denote the known position of the respective hypercube binary sharings
6. $h'_{\text{com}} \leftarrow \mathcal{H}_3(\text{salt}, M^{\min'}, M^{\max'}, (\delta_{y_j})_{j \in [\tau]}, (\delta_{z_j})_{j \in [\tau]^*})$
7. $\delta_{z_0} \leftarrow 0$
8. $h'_0 \leftarrow \mathcal{H}_0(\text{msg}, h'_{\text{com}}, \text{salt}, (\delta_{y_j}, \delta_{z_j})_{j \in [\tau]})$
9. $(\vec{\mu}, (\alpha_m)_{m \in [\tau \cdot D]}) \leftarrow \text{PRG}_0(h'_0)$
10. For $j \in [\tau]$:
 - Compute $\Delta_j^* \leftarrow \sum_{i=j \cdot D}^{(j+1) \cdot D-1} b_{i,j} \alpha_i$
 - Compute $\vec{F}_y^{(j)}(\Delta_j^*) \leftarrow \sum_{i=j \cdot D}^{(j+1) \cdot D-1} \alpha_i \left(\vec{y}_j^{\llbracket b_{i,j} \rrbracket i} + b_{i,j} \delta_{y_j} \right)$
 - Compute $Z^{(j)}(\Delta_j^*) \leftarrow \sum_{i=j \cdot D}^{(j+1) \cdot D-1} \alpha_i \left(z_j^{\llbracket b_{i,j} \rrbracket i} + b_{i,j} \delta_{z_j} \right)$
 - $a^{(j)'} \leftarrow Z^{(j)}(\Delta_j^*) \oplus \vec{F}_y^{(j)}(\Delta_j^*) \cdot \vec{\mu} \oplus b \Delta_j^*$
11. $\Delta^* \leftarrow \sum_{j=0}^{\tau-1} \Delta_j^*$
12. $\vec{F}_x(\Delta^*) \leftarrow \sum_{m=0}^{\tau \cdot D-1} \vec{x}_m^{\llbracket b_m \rrbracket} \alpha_m$
13. $\vec{F}_y(\Delta^*) \leftarrow \sum_{j=0}^{\tau-1} \vec{F}_y^{(j)}(\Delta_j^*)$
14. Compute $\Phi(\Delta^*)$ as

$$\left(\vec{u} \cdot \begin{pmatrix} \vec{F}_x(\Delta^*) \\ 1 \\ 0 \end{pmatrix} \right) \left(\vec{v} \cdot \begin{pmatrix} \vec{F}_y(\Delta^*) \\ 0 \\ 1 \end{pmatrix} \right) - \left(\vec{u} \cdot \begin{pmatrix} \vec{F}_y(\Delta^*) \\ 0 \\ 1 \end{pmatrix} \right) \left(\vec{v} \cdot \begin{pmatrix} \vec{F}_x(\Delta^*) \\ 1 \\ 0 \end{pmatrix} \right)$$
15. $A' \leftarrow \Phi(\Delta^*) - B \Delta^*$
16. $h'_1 \leftarrow \mathcal{H}_1(\text{msg}, h'_{\text{com}}, \text{salt}, (\delta_{y_j}, \delta_{z_j})_{j \in [\tau]}, h'_0, A', B, (a^{(j)'})_{j \in [\tau]}, b)$
17. If $h'_1 = h_1$ output **ACCEPT**, otherwise output **REJECT**

Fig. 12. $\text{SBC}_{\text{VOLE}}^M$ signature scheme - Verification algorithm